Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

CSE 207 Question Bank

Data Structures & Algorithms II

Topics: Graph Algorithms, Advanced Data Structures, Hashing, NP-Completeness, Coping with Hardness, String Matching Algorithms, Randomized Algorithms

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— LaTeX Typesetting

NotebookLM — Research & Extraction

Claude AI — LaTeX Typesetting

Gemini — Diagram Generation

Table of Contents

BUET · CSE 207 · Data Structures & Algorithms II

Part I

Graph Algorithms

Shortest Paths, Minimum Spanning Trees, Maximum Flow, Maximum Bipartite Matching, Stable Matching (Gale-Shapley), Graph Traversal & Properties

S1 — Graph Algorithms

Shortest Paths

Q1. Let S be a source vertex in a directed graph with non-negative edge weights. Suppose that after applying Dijkstra's algorithm, the following shortest path distances from S to all other vertices are obtained:

Vertex	S	A	B	C	D	E	F	G
Distance	0	2	4	5	7	8	9	11

Construct a directed weighted graph with non-negative integer edge weights such that running Dijkstra's algorithm from source S produces exactly the given distances, and justify that Dijkstra's algorithm produces those distances. The graph must satisfy:

(a) Every vertex must be reachable from S.

(b) The graph must contain at least two edges that are not part of any shortest path from S.

(c) The shortest path to each vertex must be uniquely determined.

(15 marks) [CSE 207 | L-2/T-1 | 2024–25]

Q2. Consider a directed weighted graph with vertices $\{A, B, C, D\}$ and edges $A \to B$ (3), $B \to C$ (2), $C \to D$ (2), $D \to B$ (-8), $A \to D$ (10), $B \to D$ (5). Determine whether the graph contains a negative weight cycle by applying the Floyd-Warshall algorithm. If a negative cycle exists, identify the vertex (or vertices) involved in the cycle. **(15 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q3. Let $c_1, c_2, \ldots, c_n$ be various currencies. For any two currencies c_i and c_j, there is an exchange rate r_{ij} which means that you can purchase r_{ij} units of currency c_j in exchange for one unit of c_i. These exchange rates usually satisfy the condition that $r_{ij} \cdot r_{ji} < 1$, so that if you start with a unit of currency c_i, change it into currency c_j and then convert back to currency c_i, you end up with less than one unit of currency c_i.

(a) Design an efficient algorithm for the following problem: Given a set of exchange rates r_{ij}, and two currencies s and t, find the most advantageous sequence of currency exchanges for converting currency s into currency t.

(b) Occasionally, there is a sequence of currencies $c_{i_1}, c_{i_2}, \ldots, c_{i_k}$ such that $r_{i_1 i_2} \cdot r_{i_2 i_3} \cdots r_{i_{k-1} i_k} \cdot r_{i_k i_1} > 1$. Give an efficient algorithm for detecting the presence of such an anomaly. (Hint: try formulating the problem using a graph.)

(15 marks) [CSE 207 | L-2/T-1 | 2023–24]

Q4. Explain the Floyd-Warshall algorithm for the all-pairs shortest paths problem. In what context is Johnson's algorithm preferable to the Floyd-Warshall algorithm? **(15 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q5. Formulate a proof on the correctness of Dijkstra's algorithm. **(13 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q6. Analyze the complexity of Johnson's algorithm for the all-pairs shortest paths problem. **(12 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q7. Develop an algorithm for finding the shortest path in a graph where all edge weights are equal. **(8 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q8. Devise an efficient algorithm for finding the shortest path where edge weights of the graph are non-equal but positive. **(12 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q9. Develop an efficient algorithm for finding the shortest path when the edge weights can be negative and the graph can have negative edge-weight cycles. **(15 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q10. The Bangladesh government is deeply concerned about addressing the severe traffic congestion issues in Dhaka. Despite numerous attempts to alleviate the problem, it remains a persistent challenge. Recognizing the need for fresh perspectives and innovative solutions, the government has turned to experts like you in the field of computer science, seeking guidance on a novel approach.

Your proposed solution is straightforward: the construction of new roads. However, the key challenge lies in ensuring the effective utilization of these newly built roads. This can only be achieved by reducing travel costs along specific routes. By doing so, the traffic will naturally redistribute throughout the network, alleviating congestion in critical areas.

In this problem, a map of Dhaka is represented as an undirected graph, where each node represents a station and each edge represents a road between two stations. These nodes are depicted as simple 2-D geometric points, and the cost of traveling along a road is directly proportional to the Euclidean distance between two nodes.

The proposed strategy involves temporarily constructing a single road, with a specific criterion for selecting the two nodes between which this new road will be built. To make the decision, whether a new road will be built between node u and node v, the following formula is employed:

$$C_{uv} = \sum (\text{PreCost}_{ij} - \text{CurCost}_{ij})$$

Here CurCost_{ij} represents the shortest path cost from any node i to any node j if a road is built between node u and node v. On the other hand, PreCost_{ij} represents the shortest path cost before the road between u and v is constructed. The pair (u, v) with the highest C_{uv} value is selected as the optimal choice.

In essence, this approach aims to identify the pair of nodes where the construction of a new road will result in the most significant improvement in the overall transportation network, effectively reducing travel costs and mitigating traffic congestion in Dhaka.
From the above problem description, answer the following:

(a) Define the input and output of the program for finding the most efficient route. **(8 marks)**

(b) Sketch the pseudo-code of the solution for finding the most efficient route. **(12 marks)**

(c) Develop the most efficient solution of the underlying algorithm that serves as the backbone of the solution. **(15 marks)**

[CSE 207 | L-2/T-2 | 2021–22]

Q11. Design an efficient algorithm for finding the length of the longest path in a dag (directed acyclic graph). **(8 marks)** **[CSE 207 | L-2/T-2 | 2020–21]**

Q12. Minimum-sum descent: Some positive integers are arranged in an equilateral triangle with n numbers in its base. The problem is to find the smallest sum in a descent from the triangle apex to its base through a sequence of adjacent numbers. Explain how the Minimum-sum descent problem can be solved by Dijkstra's algorithm.

2

5 4

1 4 7

8 6 9 6

(12 marks) **[CSE 207 | L-2/T-2 | 2020–21]**

Q13. The shortest paths to a given vertex from each other vertex of a weighted digraph is called the single-destination shortest-paths problem. Solve the single-destination shortest-path problem in a graph with nonnegative numbers assigned to its vertices in addition to the edge weights. **(12 marks)** **[CSE 207 | L-2/T-2 | 2020–21]**

Q14. Explain how to implement Warshall's algorithm without using extra memory for storing elements of the algorithm's intermediate matrices. **(10 marks)** **[CSE 207 | L-2/T-2 | 2020–21]**

Q15. Given a graph with integer edge weights between 1 and 5 (inclusive), you want to find the shortest weighted path between a pair of vertices. How would you reduce this problem to the shortest unweighted path problem, which can be solved using a Breadth First Search? **(13 marks)** **[CSE 207 | L-2/T-2 | 2019–20]**

Q16. A new startup FastRoute wants to route information along a path in a network. Each vertex represents a router and each edge in the network has a bandwidth. Design an algorithm to find the path with maximum bandwidth from any source s to any destination t. The bandwidth of a path is the minimum of the bandwidths of the edges on that path; the minimum edge is the bottleneck. Explain how to modify Dijkstra's algorithm to do this. Justify your answer. **(15 marks)** **[CSE 207 | L-2/T-2 | 2019–20]**

Q17. Show how to express the single-source shortest-paths problem as a product of matrices and a vector. Describe how evaluating this product corresponds to a Bellman-Ford-like algorithm. **(10 marks)** **[CSE 207 | L-2/T-2 | 2019–20]**

Q18. Give an efficient algorithm to find the length (number of edges) of a minimum-length negative-weight cycle in a graph. **(10 marks)** **[CSE 207 | L-2/T-2 | 2019–20]**

Q19. You are given a strongly connected directed graph $G = (V, E)$ with positive edge weights along with a particular node $v_0 \in V$. Give an efficient algorithm for finding shortest paths between all pairs of nodes, with the one restriction that these paths must all pass through v_0. **(12 marks)** **[CSE 207 | L-2/T-2 | 2019–20]**

Q20. Dijkstra's algorithm for the shortest path problem is shown below. What would be the time-complexity of the algorithm (show line by line analysis) for an input graph G if adjacency list is used for graph representation and an ordinary array is used for storing $d[\,]$ values.

```
Dijkstra (G)
1  Q = V[G];
2  for each u in Q
3     d[v] = INF;
4  d[s] = 0; S = {};
5  while (Q != {})
6     u = ExtractMin(Q);
7     S = S U {u};
8     for each v in u->Adj[]
```

```
9          if (v in Q and d[v] > d[u] + w(u,v))
10            d[v] = d[u] + w(u,v);
```

(15 marks) [CSE 207 | L-2/T-2 | 2018–19]

Q21. Johnson's Algorithm makes clever use of Bellman-Ford and Dijkstra's Algorithms to do All-Pairs-Shortest-Paths efficiently on sparse graphs. Explain the basic clever idea of Johnson's Algorithm. **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q22. How can BFS be adapted to solve the single-source shortest path problem in an edge-weighted graph with unequal weights? Explain. **(8 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q23. Explain the intuitive idea behind the Bellman-Ford algorithm. How can you detect a negative cycle using Bellman-Ford? **(6+2 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q24. Write Dijkstra's algorithm for the single-source shortest path problem. Prove its correctness. Analyze the time complexity of the algorithm. What would be the running time if a Fibonacci heap is used instead of a binary heap? **(5+5+7+5 marks)** [CSE 207 | L-2/T-2 | 2017–18]

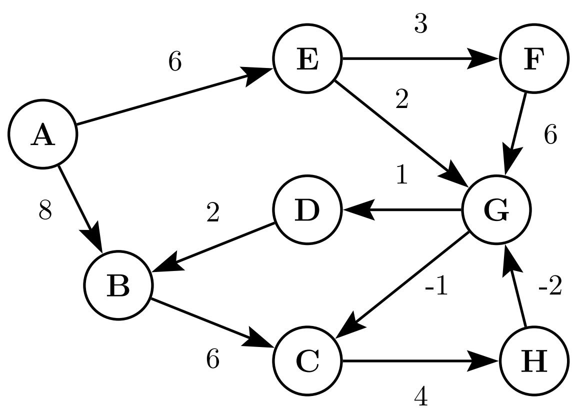
Q25. Write the properties satisfied by shortest paths of a graph. Show by example that Dijkstra's algorithm gives wrong results for a graph with negative-weight edges. Write an algorithm that finds shortest paths in a DAG and analyze its time complexity. **(15 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q26. Prove that the Bellman-Ford algorithm returns TRUE if the input graph contains no negative-weight cycles reachable from the source, and FALSE otherwise. **(10 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q27. State and prove the convergence property of edge relaxation in the context of shortest paths. **(8 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q28. Provide a correctness proof of Dijkstra's algorithm. **(10 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q29. Apply the Bellman-Ford algorithm to find the shortest paths. Show all relaxation steps.



(10 marks) [CSE 207 | L-2/T-2 | 2014–15]

Q30. Prove that the component graph of a directed graph is a DAG. **(6 marks)** [CSE 207 | L-2/T-2 | 2012–13]

Q31. Assume that a breadth-first tree has already been computed with BFS on a graph G with source vertex s. Write an algorithm to print the vertices on a shortest path from s to v. **(8 marks)** [CSE 207 | L-2/T-2 | 2012–13]

Q32. State and prove the convergence property of shortest paths. **(12 marks)** [CSE 207 | L-2/T-2 | 2012–13]

Q33. Write the Bellman-Ford algorithm. Analyze the time complexity and correctness of the Bellman-Ford algorithm. **(20 marks)** [CSE 207 | L-2/T-2 | 2012–13]

Q34. Does Dijkstra's algorithm terminate if it is applied on a graph G that contains a negative-weight cycle? Why or why not? Justify your answer. **(6 marks)** [CSE 207 | L-2/T-2 | 2012–13]

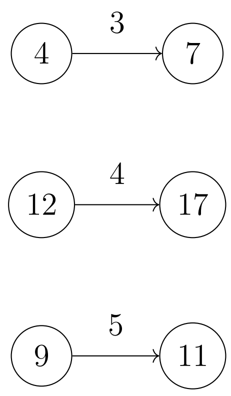
Q35. What does $d[v]$ represent at the end of the execution of Algorithm 1, if the algorithm is applied on an unweighted directed acyclic graph (DAG) G with a source s? What is the runtime complexity in terms of $|V|$ and $|E|$ of Algorithm 1?

Algorithm: LONGESTPATH-DAG(G)

Compute a topological sorting of G
for each $v \in V[G]$ **do**
  $d[v] \leftarrow 0$
end for
for each u in topologically sorted order **do**
  for each $v \in Adj[u]$ **do**
    if $d[v] < d[u] + 1$ **then**
      $d[v] \leftarrow d[u] + 1$
    end if
  end for
end for

(6+6 marks) [CSE 207 | L-2/T-2 | 2012–13]

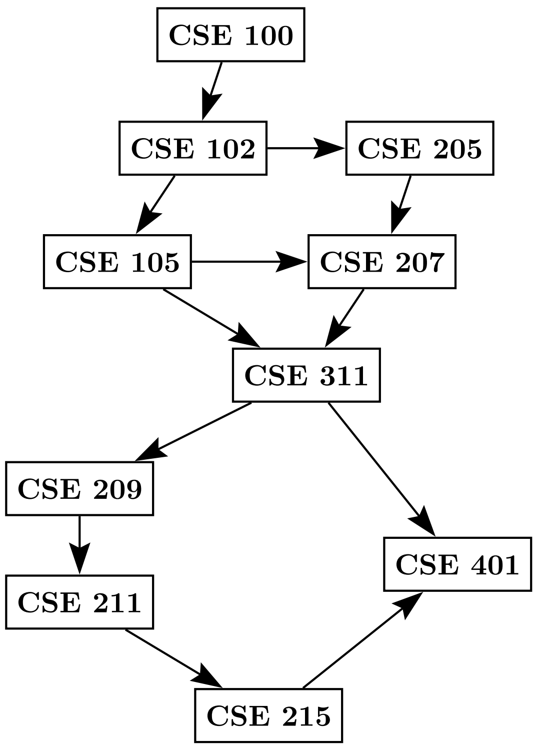
Q36. In the figure below, the shortest path estimate of each vertex appears within the vertex and the distance to traverse from a vertex to another appears as the weight of the corresponding directed edge. Show the updated shortest path estimates of vertices after relaxing the following edges.



(6 marks) [CSE 207 | L-2/T-2 | 2012–13]

Q37. Explain how Dijkstra's algorithm can be implemented using a priority queue. Find the running time for this algorithm. **(15 marks)** [CSE 207 | L-2/T-2 | 2011–12]

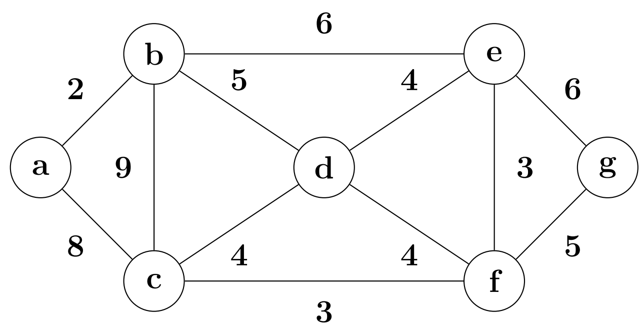
Q38. Find the topological order of courses for this given prerequisite graph. **(10 marks)** [CSE 207 | L-2/T-2 | 2006–07]



Minimum Spanning Trees

Q1. Consider the following undirected weighted graph. Vertices: $\{A, B, C, D, E, F\}$. Edges: A–B (1), A–C (3), A–D (4), B–C (2), B–D (6), C–D (5), C–E (7), D–E (8), E–F (9), D–F (10), C–F (11). Determine whether the minimum spanning tree (MST) of this graph is unique. Justify your answer rigorously using MST properties such as the cut property or cycle property. If the MST is not unique, construct at least two distinct MSTs with equal total cost. If it is unique, provide a formal justification explaining why no alternative MST with the same total weight can exist. **(20 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q2. Draw a minimum spanning tree of the graph shown in the figure using (i) Prim's algorithm, and (ii) Kruskal's algorithm. Write the order of the edges in each case (whenever there is a choice of vertices, always use alphabetical ordering).



(12 marks) [CSE 207 | L-2/T-1 | 2023–24]

Q3. Prove (i) the optimal sub-structure property and (ii) the greedy choice property of the Minimum Spanning Tree (MST) problem. **(12 marks)** [CSE 207 | L-2/T-2 | 2021–22]

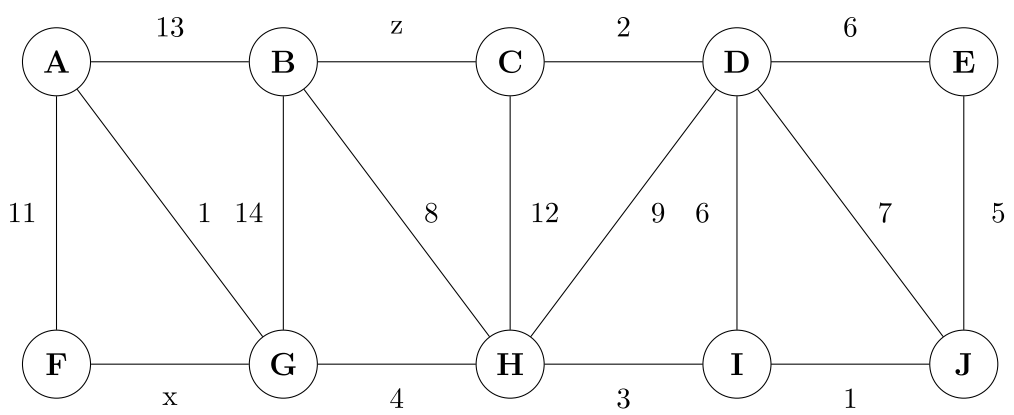
Q4. Develop a solution of the MST problem using disjoint set / union-find data structure. Use the array implementation to maintain the union-find data structure. **(15 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q5. Let T be a minimum spanning tree of graph G obtained by Prim's algorithm. Let G_{new} be a graph obtained by adding to G a new vertex and some edges, with weights, connecting the new vertex to some vertices in G. Can you construct a minimum spanning tree of G_{new} by adding one of the new edges to T? If your answer is yes, explain how; if your answer is no, explain why not. **(15 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q6. Prove that if all edge weights of a graph are positive, then any subset of edges that connects all vertices and has minimum total weight must be a tree. Give an example to show that the same conclusion does not follow if we allow some weights to be nonpositive. **(12 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Q7. Show that for each minimum spanning tree T of G, there is a way to sort the edges of G in Kruskal's algorithm so that the algorithm returns T. (Kruskal's algorithm can return different spanning trees for the same input graph G, depending on how it breaks ties when the edges are sorted.) **(10 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Q8. Prove that an optimal minimum spanning tree is composed of optimal minimum spanning subtrees. Draw a minimum spanning tree of the graph G shown below such that the minimum spanning tree contains the edges with weights x and z.

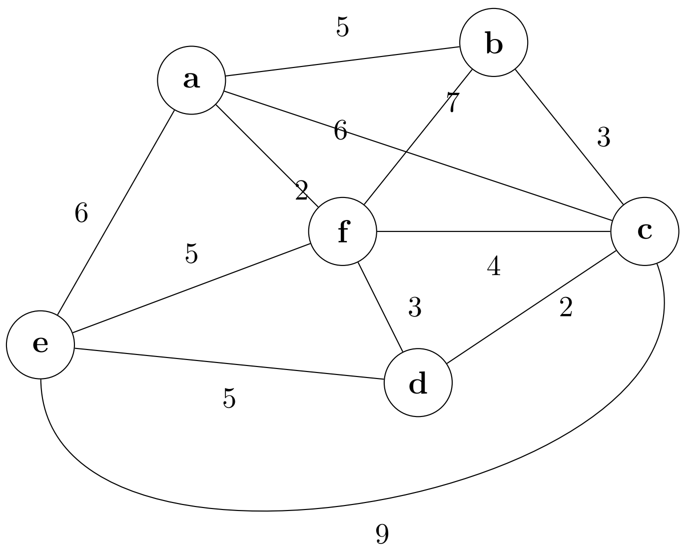


(15 marks) [CSE 207 | L-2/T-2 | 2018–19]

Q9. Write down an algorithm for constructing a mincost arborescence in a digraph from a given root vertex r. Apply the algorithm to the following digraph, showing every step in separate diagraphs: $ra(7), rx(2), ax(1), xy(5), ya(3), xc(6), cb(4), bx(2), yb(2)$ (weights in parentheses). **(15 marks)** [CSE 203 | L-2/T-1 | 2017–18]

Q10. What is a spanning tree of a graph? Write Kruskal's algorithm for finding a minimum spanning tree of an edge-weighted graph. Analyze the time complexity of the algorithm by suggesting appropriate data structures. **(3+5+7 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q11. Find a minimum spanning tree using Prim's algorithm, showing every step.



(10 marks) [CSE 207 | L-2/T-2 | 2017–18]

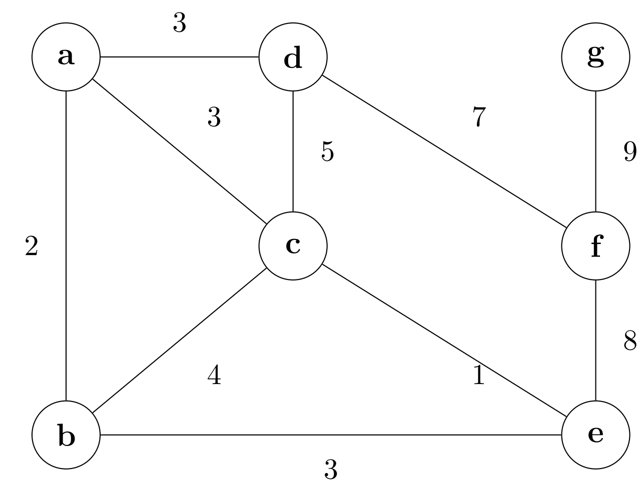
Q12. Let G be a weighted connected graph and let V_1, V_2 be a partition of its vertices into two disjoint non-empty sets. Let e be an edge in G with minimum weight among those with one endpoint in V_1 and the other in V_2. Show that there is a minimum spanning tree T of G containing e. **(10 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q13. Write Prim's algorithm that computes a minimum spanning tree of a graph. Analyze its time complexity. **(15 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q14. Write the correctness proof of Kruskal's algorithm for finding a minimum spanning tree. **(10 marks)** [CSE 207 | L-2/T-2 | 2016–17]

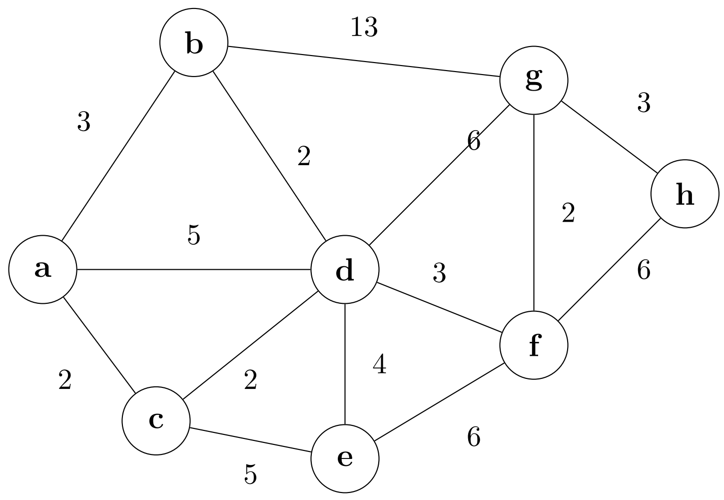
Q15. There is a network of roads $G = (V, E)$ connecting a set of cities V. Each road in E has an associated length l_e. There is a proposal to add one new road to this network, and there is a list E' of pairs of cities between which the new road can be built. Each such potential road $e' \in E'$ has an associated length. As a designer for the public works department you are asked to determine the road $e' \in E'$ whose addition to existing network G would result in the maximum decrease in the driving distance between two fixed cities s and t in the network. Give an algorithm for solving this problem. Your algorithm should run in $O((V + E + E') \log V)$ time. **(15 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Q16. Find a minimum spanning tree by running (i) Kruskal's algorithm and (ii) Prim's algorithm. Show the order of edges selected in each case (use alphabetical ordering when there is a tie). **(15 marks)** [CSE 207 | L-2/T-2 | 2015–16]



Q17. Let $G = (V, E)$ be a connected undirected graph with weight function w on E. Let $A \subseteq E$ be included in some MST for G, let $(S, V - S)$ be any cut that respects A, and let (u, v) be a light edge crossing $(S, V - S)$. Prove that (u, v) is safe for A. **(10 marks)** [CSE 207 | L-2/T-2 | 2014–15]

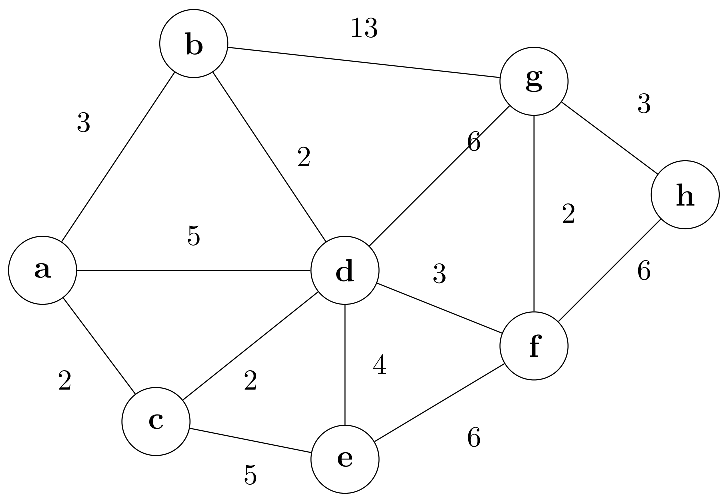
Q18. Find the light and respectful crossing edges for each step of Kruskal's method for building the MST.



(12 marks) [CSE 207 | L-2/T-2 | 2014–15]

Q19. What is the cost of finding and joining (union) of sets in Kruskal's method? **(8 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q20. For the graph below, show the value of u, $v.key$, Q for the following algorithm (show each step):



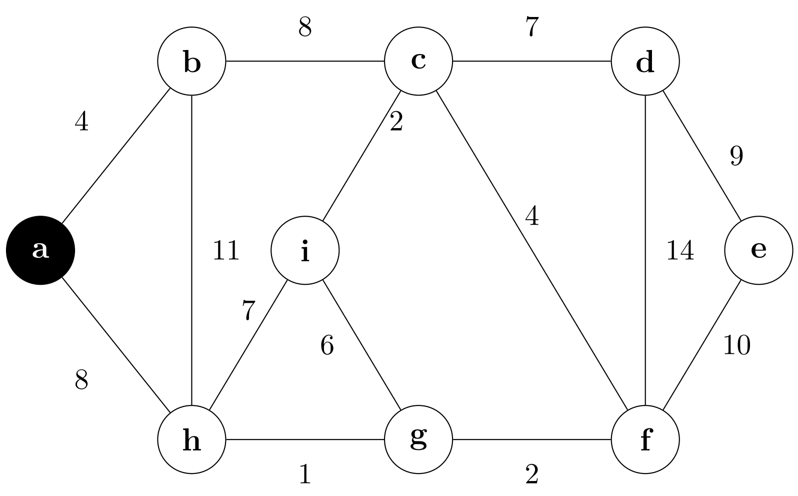
(10 marks) [CSE 207 | L-2/T-2 | 2014–15]

Q21. *GreedyCell*, a mobile phone operator, wants to place cell towers (base stations) along a long straight country road, to provide network coverage to n houses sparsely scattered along the road. The distance of these houses from the start of the road are, in miles and in increasing order, $d_1, d_2, \ldots, d_n$. Each cell tower has coverage of R miles, i.e., if a house is within R miles of a tower, it would get service from that tower. Now give an efficient algorithm for finding a placement of cell towers that uses as few cell towers as possible to provide coverage to all the n houses.**(12 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q22. Given a directed graph with edge list and weights $AB(5), BC(2), AC(4), AD(6),$ $CD(3), DB(1), DE(6), EB(3), EF(3), FB(4)$, construct a minimum-cost arborescence rooted at A. **(10 marks)** [CSE 207 | L-2/T-2 | 2013–14]

Q23. Find a minimum spanning tree using both Kruskal's algorithm and Prim's algorithm (starting from A) for the graph with edges: $AB(5), BC(2), AC(4), AD(6), CD(3),$ $DB(1), DE(6), EB(3), EF(3), FB(4)$. **(15 marks)** [CSE 207 | L-2/T-2 | 2013–14]

Q24. If Prim's algorithm is applied on a given graph, which will be the fifth edge added to the spanning tree? Start from vertex a. **(10 marks)** [CSE 207 | L-2/T-2 | 2012–13]



Q25. State and prove the "cut property" and the "cycle property". Explain with necessary examples how these properties are useful in solving the minimum spanning tree problem. **(15 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Q26. How is the minimum cost arborescence problem different from the minimum spanning tree problem? Construct a minimum cost arborescence for the graph with edges $AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), FB(5), CE(4), DE(6)$, $FD(3)$, with edge orientations from the first vertex to the second. Assume source vertex is A. **(5+5 marks)** [CSE 207 | L-2/T-2 | 2006–07]

Maximum Flow

Q1. A central warehouse supplies relief packages to three depots D_1, D_2, D_3. The depots distribute packages to four affected zones Z_1, Z_2, Z_3, Z_4. Each depot has a daily sending capacity: $D_1 = 15$, $D_2 = 20$, $D_3 = 10$. Each zone has a receiving limit: $Z_1 = 10$, $Z_2 = 15$, $Z_3 = 10$, $Z_4 = 10$. Distribution routes: $D_1 \to \{Z_1, Z_2\}$, $D_2 \to \{Z_1, Z_3, Z_4\}$, $D_3 \to \{Z_2, Z_4\}$. The warehouse supply is unlimited. Determine the maximum number of relief packages that can be delivered per day. Model the situation as a flow network by clearly defining the source, sink, intermediate nodes, and edge capacities and apply Ford-Fulkerson's algorithm. **(20 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q2. Derive a proof for the max-flow min-cut theorem. **(15 marks)** [CSE 207 | L-2/T-1 | 2023–24, 2022–23]

Q3. Define the Max-flow problem. **(8 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q4. Consider a car parking problem scenario, where you need to match each of the cars with a parking spot. The cars and the available parking spots are located at different parts of the city and can only accessible through a road network graph, G. Let A be the set of cars and B be the set of parking slots. The cost of a car, $a \in A$ to reach to a parking slot, $b \in B$ is the shortest distance (on the graph G) between a and b. If the spot is unreachable by a car, we can assign it an infinite cost.

(a) Formulate the car parking assignment problem using the Max-flow/min-cut framework. **(12 marks)** [CSE 207 | L-2/T-2 | 2021–22]

(b) Develop an efficient solution of the above problem to find a match for every car so that the overall cost is minimized. **(15 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q5. You want to increase the maximum flow of a network as much as possible, but you are only allowed to increase the capacity of one edge. How do you find such an edge? (Give pseudocode). You may assume the existence of algorithms to compute max flow min cut. What's the running of your algorithm? **(10 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q6. The edge connectivity of an undirected graph is the minimum number k of edges that must be removed to disconnect the graph. For example, the edge connectivity of a tree is 1, and the edge connectivity of a cyclic chain of vertices is 2. Show how to determine the edge connectivity of an undirected graph $G = (V, E)$ by running a maximum-flow algorithm on at most $|V|$ flow networks, each having $O(V)$ vertices and $O(E)$ edges. **(15 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q7. Ad-hoc networks are made up of cheap, low-powered wireless devices, which can communicate within a limited range. Consider an ad-hoc wireless network where any two devices can communicate if and only if the distance between them is at most D. To improve reliability, we require each device x to have k potential backup devices, all within distance D of x; we call these k devices the backup set of x. Also, we do not want any device to be in the backup set of more than b devices, say; otherwise, a single failure might affect a large fraction of the network.

Suppose we are given the communication distance D, parameters b and k, and an array $d[1 \dots n, 1 \dots n]$ of distances, where $d[i, j]$ is the distance between device i and device j. The problem is to design and analyze an algorithm that either computes a backup set of size k for each of the n devices, such that no device appears in more than b backup sets, or correctly reports that no good collection of backup sets exists. Now answer the following questions:

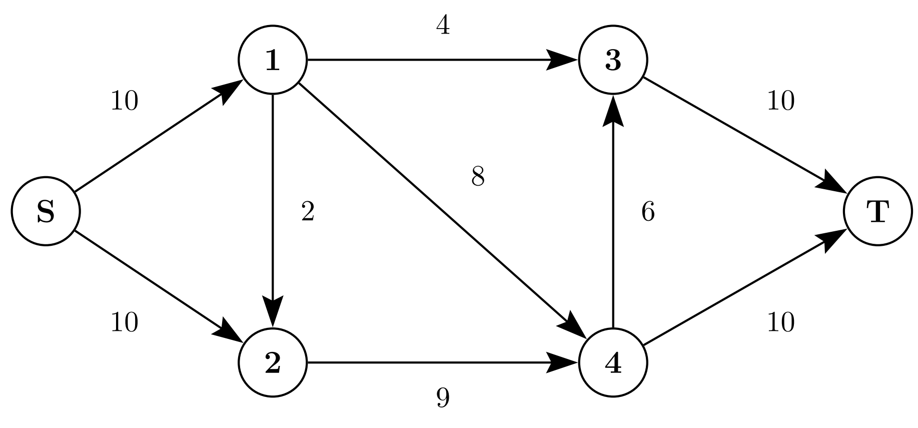
(a) Reduce this problem to a max-flow problem. Describe a max-flow problem (a directed, capacitated graph G, a source and sink pair) such that a good collection of backup sets exist if and only if the max-flow in G has a certain size. What is this size?

(b) Prove the correctness of your reduction. Show that a good collection of backup sets exist if and only if the max-flow in G has a certain size.

(c) Give an algorithm to solve this max-flow problem. What is the running time of this algorithm as a function of n, k and b?

(23 marks) [CSE 207 | L-2/T-2 | 2019–20]

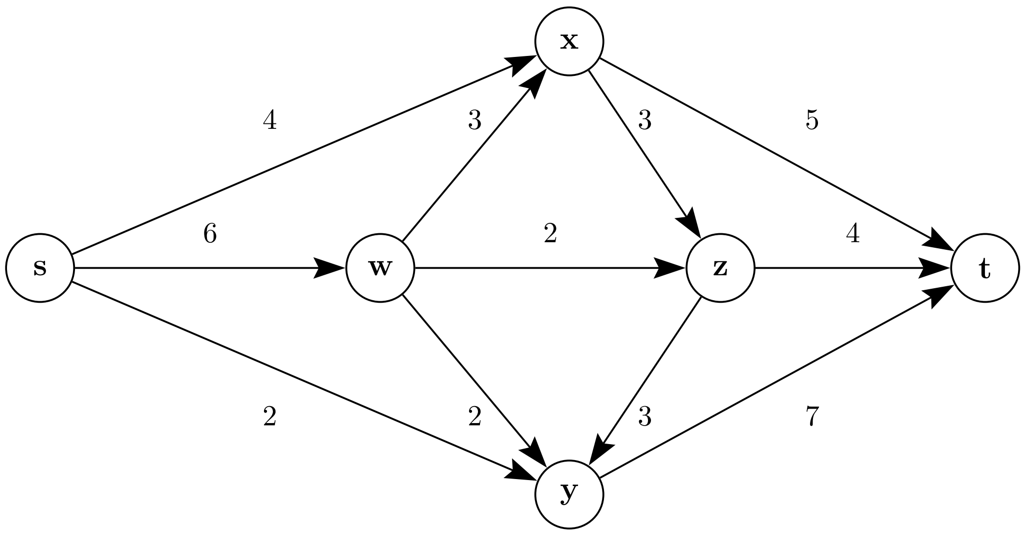
Q8. For the flow network shown in the following figure, find the value of the maximum flow by drawing residual networks in each step. Finally, draw the maximum flow network. Also, identify the min-cut of the maximum flow network.



(15 marks) [CSE 207 | L-2/T-2 | 2018–19]

Q9. Define a flow network. What is an augmenting path in a flow network? Explain with illustrative examples. **(3+3 marks)** [CSE 207 | L-2/T-2 | 2017–18]

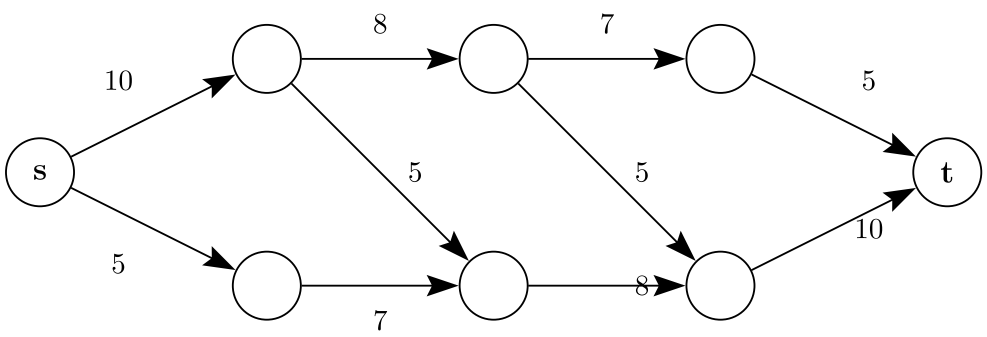
Q10. Find the maximum flow and the minimum cut using the Ford-Fulkerson algorithm.The vertex s is the source and the vertex t is the sink of the flow network. Integers represent the flow capacities of the edges.



(10 marks) [CSE 207 | L-2/T-2 | 2017–18]

Q11. Describe the minimum path length augmentation method of Edmonds and Karp and explain how it improves the time complexity of the Ford-Fulkerson algorithm. **(10 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q12. Find the value of the maximum flow by drawing residual networks at each step. Draw the maximum flow network and identify the min-cut.



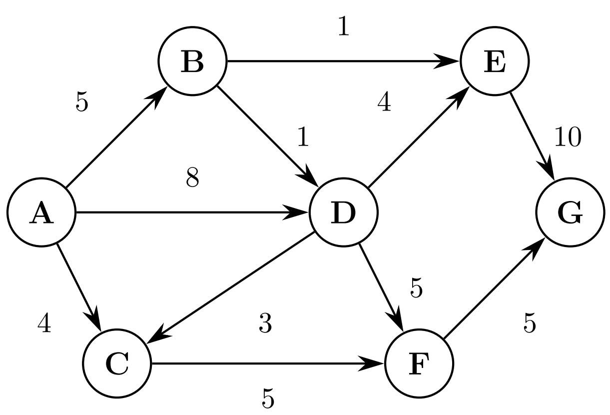
(15 marks) [CSE 207 | L-2/T-2 | 2016–17]

Q13. What is a flow network? Explain how network flow algorithms allow us to find the maximum bipartite matching. **(10 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q14. Write the Edmonds-Karp algorithm that solves the maximum-flow problem. Analyze its time complexity for integral capacities. **(10 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Q15. Explain how network flow algorithms allow us to find the maximum bipartite matching. **(10 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Q16. A traffic engineer needs to decide which roads to widen to allow maximum traffic from A to G. Which roads are to be widened?

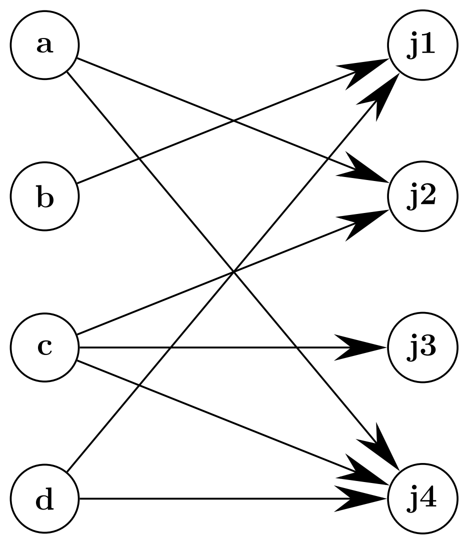


(12 marks) [CSE 207 | L-2/T-2 | 2014–15]

Q17. Define and explain the following terms in a network flow model: super sink node, augmented path, residual capacity, super source node. **(8 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q18. What is the pitfall of Ford-Fulkerson's max-flow min-cut method? How can it be improved? **(8 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q19. Find the maximum job assignment using a maximum flow algorithm.

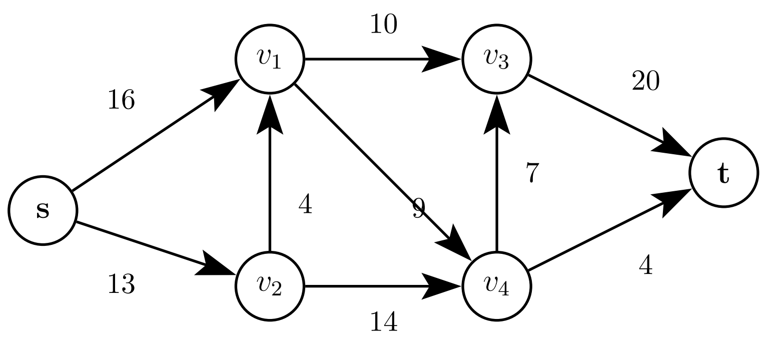


(10 marks) [CSE 207 | L-2/T-2 | 2014–15]

Q20. What is a flow network? What is the maximum-flow problem? Explain how one can convert a multiple-source, multiple-sink maximum-flow problem into a problem with a single source and a single sink. **(4+2+4=10 marks)** [CSE 207 | L-2/T-2 | 2011–12]

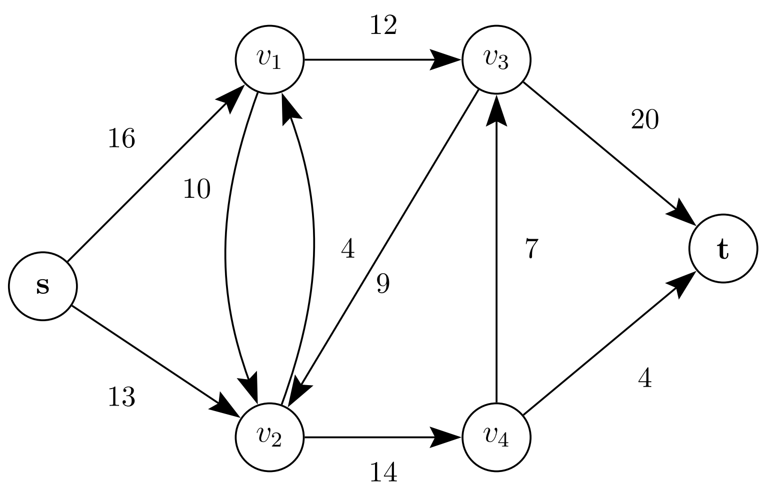
Q21. Write the Ford-Fulkerson algorithm for solving the maximum-flow problem, and analyze the time complexity of the algorithm. Explain how the Edmonds-Karp algorithm improves the bound of the Ford-Fulkerson algorithm. **(10+5=15 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Q22. State the max-flow min-cut theorem in its simplest form. What is the main argument to prove the theorem? Find a min-cut and its capacity. Find the flow in that cut and the maximum flow in the network.



(17 marks) [CSE 207 | L-2/T-2 | 2009–10]

Q23. Find the maximum flow and the minimum cut. Find also the maximum flow in the minimum cut.



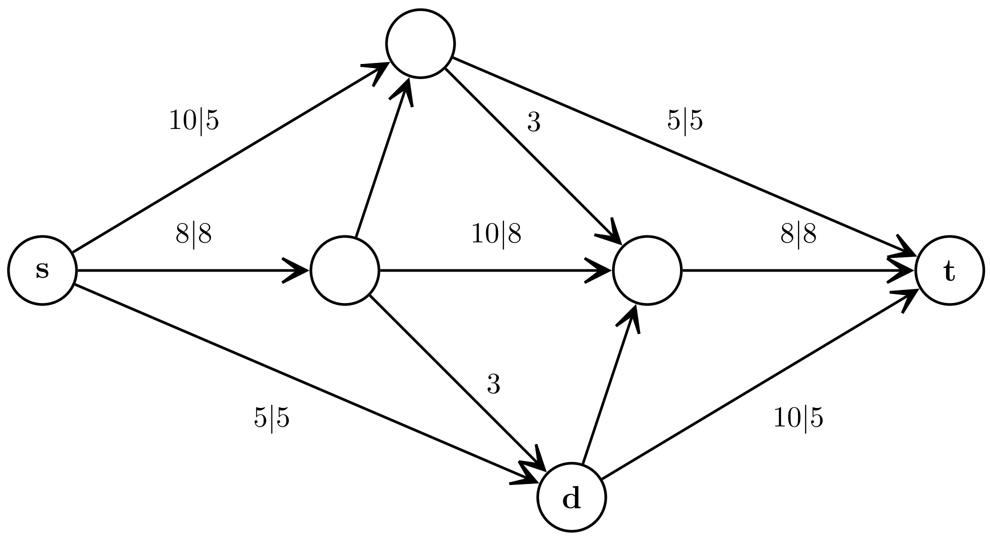
(12 marks) [CSE 207 | L-2/T-2 | 2008–09]

Q24. Discuss how a maximum flow algorithm can be used to determine the maximum number of disjoint paths in directed and undirected graphs. **(15 marks)** [CSE 207 | L-2/T-2 | 2007–08]

Q25. Figure shows a flow network on which s-t flow has been computed. The capacity of each edge appears as a label next to the edge, and the numbers in boxes give the amount of flow sent on each edge. Edges without boxed numbers have no flow on them.

(a) What is the value of this flow? Is this a maximum (s, t) flow on this graph?

(b) Find a minimum s-t cut in the flow network pictured in Figure 1, and also say what its capacity is.



(20 marks) [CSE 207 | L-2/T-2 | 2007–08]

Q26. There is a wireless network of nodes A, B, C, D, E, F, G and H. Capacity of the wireless links are given in the following format: "-" indicates there is no link between those nodes. What is the maximum data flow, if node A sends data to node H? What is the capacity of the optimal cut? (Use Ford-Fulkerson method and show the steps.) **(10+5=15 marks)**

	A	B	C	D	E	F	G	H
A	-	10	5	15	-	-	-	-
B	-	-	4	-	9	15	-	-
C	-	-	-	4	-	8	-	-
D	-	-	-	-	-	-	30	-
E	-	-	-	-	-	15	-	10
F	-	-	-	-	-	-	15	10
G	-	-	6	-	-	-	-	10
H	-	-	-	-	-	-	-	-

[CSE 207 | L-2/T-2 | 2006–07]

Q27. How can the Ford–Fulkerson method be used to find a maximum matching? Explain with a simple example. **(10 marks)** [CSE 207 | L-2/T-2 | 2006–07]

Maximum Bipartite Matching

Q1. Formulate the problem of maximum bipartite matching. Give two examples of real-life problems that can be solved using maximum bipartite matching. **(12 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q2. Develop a solution for the maximum bipartite matching problem. What is the Edmonds-Karp optimization for the Ford-Fulkerson algorithm? **(15 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q3. Explain with an example how to convert a bipartite graph G to a flow network G' such that a maximum bipartite matching M in G can be computed by finding a maximum flow f in G'. Prove that there is a maximum matching M in G if and only if there is a maximum flow f in G'. **(17 marks)** [CSE 207 | L-2/T-2 | 2009–10]

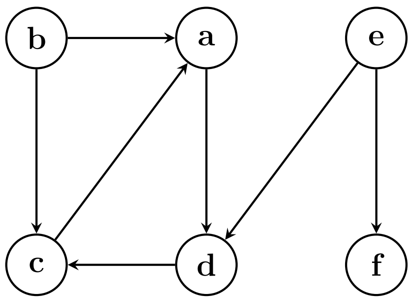
Q4. Explain with an example how to convert a bipartite graph G to a flow network G' such that a maximum bipartite matching M in G can be computed by finding a maximum flow f in G'. Prove that there is a maximum matching M in G if and only if there is a maximum flow f in G'. **(17.5 marks)**

Stable Matching (Gale-Shapley)

Q1. In a stable matching problem (**Gale-Shapley algorithm**) each participant has a true preference order. Now consider a woman w. Suppose w prefers man m to m', but both m and m' are low on her list of preferences. Can it be the case that by switching the order of m and m' on her list of preferences (i.e., by falsely claiming that she prefers m' to m) and running the algorithm with this false preference list, w will end up with a man m'' that she truly prefers to both m and m'? Give proof to support your answer. **(10 marks)** [CSE 207 | L-2/T-1 | 2020–21]

Graph Traversal & Properties

Q1. Classify the edges of the given graph after performing a Depth-First Search (DFS). If multiple nodes are available to explore at any point, always choose the node that comes first in alphabetical ordering. Start the DFS traversal from node a. (Possible classes: tree-edge, forward edge, back edge, and cross edge.)



(10 marks) **[CSE 207 | L-2/T-1 | 2023–24]**

Q2. Gotham city is in grave danger. Batman must suit up with the following constraints:

(a) He must wear the bodysuit first.

(b) Both utility belt and armor plate can be worn in any order on top of the bodysuit.

(c) The grappling gun and smoke bombs can be attached only after the utility belt is worn.

(d) The cape has to be mounted on top of the armor plate.

(e) The glider wing must be attached to both the armor plate and the utility belt.

First, draw a dependency graph of all items. Then, recommend a valid ordering of the items to help Batman get ready quickly but properly. **(12 marks)** **[CSE 207 | L-2/T-1 | 2023–24]**

Q3. Mention the runtimes of the following algorithms:

(a) Kruskal's algorithm without Disjoint Set Union (DSU),

(b) Kruskal's algorithm with DSU,

(c) Ford-Fulkerson algorithm,

(d) Edmonds-Karp algorithm.

(8 marks) **[CSE 207 | L-2/T-1 | 2023–24]**

Q4. Consider a peculiar city with N towns and M bidirectional roads (connecting towns) classified into three types. This city is peculiar due to unconventional regulations regarding road usage: men are permitted to travel only on roads of types 1 and 3, while women are permitted to travel only on roads of types 2 and 3 exclusively. The Mayor of the city intends to renovate the city by reconstructing some roads, with the condition that the normal routine of the citizens should not be hampered.

Now, you are hired as a computer scientist to answer an important question: What is the maximum number of roads that can be renovated (remain closed for traffic during renovation) while ensuring that the city remains connected for both men and women? The city is said to be connected if it is still possible to travel form any town X to any town Y using the existing roads.

(a) Formulate the problem as a graph problem.

(b) Develop the solution using a well-known graph algorithm.

(c) Write the pseudo-code of the solution.

(23 marks) **[CSE 207 | L-2/T-1 | 2022–23]**

Q5. Define and distinguish the two basic properties of the greedy algorithm. **(8 marks)** **[CSE 207 | L-2/T-2 | 2021–22]**

Q6. **Jack Straws:** In the game of Jack Straws, a number of plastic or wooden "straws" are dumped on the table and players try to remove them one by one without disturbing the other straws. Here, we are only concerned with whether various pairs of straws are connected by a path of touching straws. Given a list of the endpoints for $n > 1$ straws (as if they were dumped on a large piece of graph paper), determine all the pairs of straws that are connected. Note that touching is connecting, but also that two straws can be connected indirectly via other connected straws. **(15 marks)** **[CSE 207 | L-2/T-2 | 2020–21]**

Q7. **Rumor spreading:** There are n people, each in possession of a different rumor. They want to share all the rumors with each other by sending electronic messages. Assume that a sender includes all the rumors he or she knows at the time the message is sent and that a message may only have one addressee. Design a greedy algorithm that always yields the minimum number of messages they need to send to guarantee that every one of them gets all the rumors. **(13 marks)** **[CSE 207 | L-2/T-2 | 2020–21]**

Q8. A vertex cover of a graph $G = (V, E)$ is a subset of vertices $S \subseteq V$ that includes at least one endpoint of every edge in E. Give a linear-time algorithm for the following task.
Input: An undirected tree $T = (V, E)$
Output: The size of the smallest vertex cover of T. **(13 marks)** **[CSE 207 | L-2/T-2 | 2019–20]**

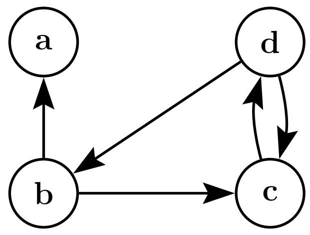
Q9. The reverse of a directed graph $G = (V, E)$ is another directed graph $G_R = (V, E_R)$ on the same vertex set, but with all edges reversed; that is, $E_R = \{(v, u) : (u, v) \in E\}$. Give a linear-time algorithm for computing the reverse of a graph in adjacency list format. **(10 marks)** **[CSE 207 | L-2/T-2 | 2019–20]**

Q10. The strength of a path in an edge-weighted graph G is the minimum weight among its edges. The all-pairs strongest path problem asks to find the strongest path between every pair of vertices. Design an algorithm to solve this problem. **(5 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q11. Given an undirected graph $G = (V, E)$, give a linear-time algorithm to check whether there is a cycle of odd length in the graph. Briefly justify its correctness. **(15 marks)** [CSE 207 | L-2/T-2 | 2015–16]

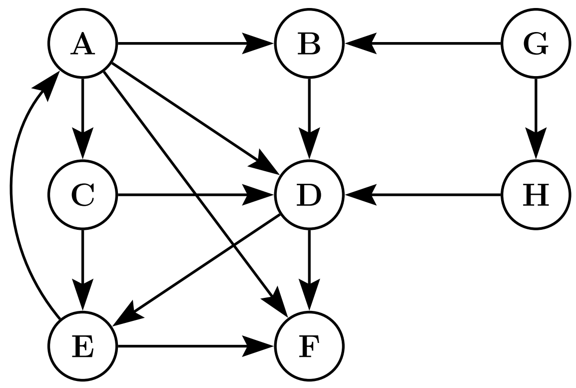
Q12. Suppose a CS curriculum consists of n mandatory courses. The prerequisite directed graph G has a node for each course and an edge from v to w if v is a prerequisite for w. Give a linear-time algorithm that computes the minimum number of semesters necessary to complete the curriculum (a student may take any number of courses per semester). **(15 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Q13. Find the transitive closure. Show the matrices computed at each step. What is the cost to find the transitive closure? **(10 marks)** [CSE 207 | L-2/T-2 | 2014–15]

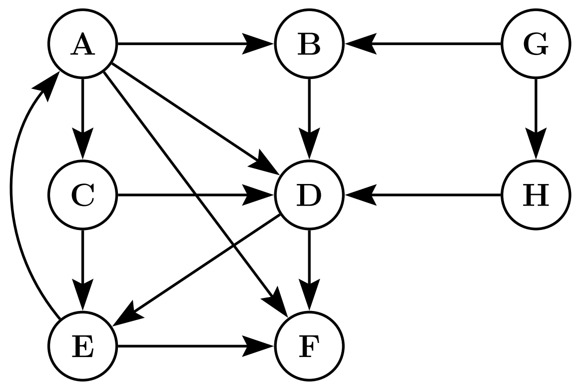


Q14. Let C and C' be distinct strongly connected components in a directed graph $G = (V, E)$. Suppose there is an edge $(u, v) \in E$ where $u \in C$ and $v \in C'$. Prove that $f(C) > f(C')$, where $f(U)$ denotes the latest finishing time of any vertex in U. **(10 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q15. Find the different types of edges if DFS is applied from vertex A. Discuss the edges' properties with respect to the graph. **(6 marks)** [CSE 207 | L-2/T-2 | 2014–15]



Q16. Show the topological sorting order of a given graph with explanation of discovery and finishing times of node exploration. **(6 marks)** [CSE 207 | L-2/T-2 | 2014–15]

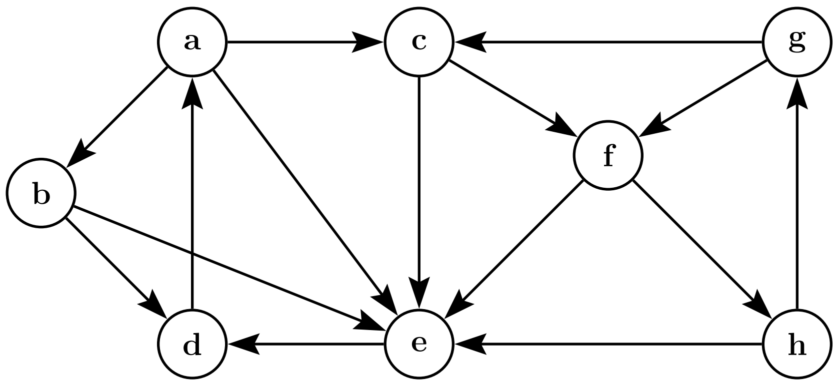


Q17. Write the Bellman-Ford algorithm. Analyze the time complexity and correctness of the Bellman-Ford algorithm. **(20 marks)** [CSE 207 | L-2/T-2 | 2012–13]

Q18. Write a linear-time algorithm to compute the strongly connected components of a directed graph $G = (V, E)$ using two depth-first searches. Give an example to show every step of the algorithm. **(8+5 marks)** [CSE 207 | L-2/T-2 | 2012–13]

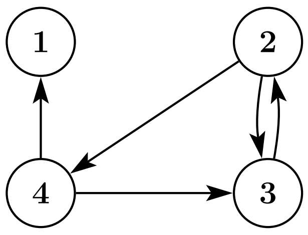
Q19. Write an algorithm for DFS traversal of a given graph. Analyze the time complexity of the algorithm. **(10+5=15 marks)** [CSE 203 | L-2/T-1 | 2011–12]

Q20. Write the sequences of the vertices if you explore the given graph G using BFS and DFS. Classify the edges of G for your DFS traversal. **(4+6=10 marks)** [CSE 203 | L-2/T-1 | 2011–12]



Q21. Derive with explanation the recursive equation for the Floyd-Warshall algorithm for all-pairs shortest-path estimates in a directed graph G. Based on this equation, write pseudocode for Floyd-Warshall. Convert this recursive equation to compute the transitive closure of G. **(17 marks)** **[CSE 207 | L-2/T-2 | 2009–10]**

Q22. For the given graph G, find the transitive closure using the Floyd-Warshall algorithm. Give the algorithm also. How can the complexity of Floyd-Warshall's all-pairs shortest paths be improved? **(10+5+10 marks)** **[CSE 207 | L-2/T-2 | 2008–09]**



Q23. How can the complexity of Floyd-Warshall's algorithm for "all pairs shortest paths" be improved? **(10 marks)**

BUET · CSE 207 · Data Structures & Algorithms II

Part II

Advanced Data Structures

Balanced Binary Search Trees (AVL Trees), Red-Black Trees, Splay Trees, Skip Lists, Advanced Heaps (Binomial Heaps), Advanced Heaps (Fibonacci Heaps)

S2 — Advanced Data Structures

Balanced Binary Search Trees (AVL Trees)

Q1. After inserting a key into an AVL tree, a node A becomes unbalanced such that its balance factor becomes $+2$, while its left child B has a balance factor of -1. Based on this information, determine the exact type of imbalance that has occurred and describe the precise sequence of rotations required to restore balance. Your answer must include a clear transformation of the subtree rooted at A, showing its structure before and after applying the rotations, along with the final balance factors of all nodes involved. **(10 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q2. Construct an AVL tree with the minimum number of nodes when the tree height is 5. **(10 marks)** [CSE 207 | L-2/T-1 | 2024–25]

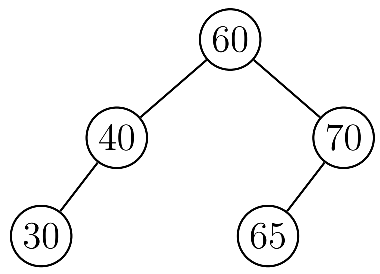
Q3. Prove that the height of an AVL tree with n elements is $O(\log n)$. **(10 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q4. Perform the following operations sequentially on an initially empty AVL tree. Show the intermediate trees after each step: (i) insert 15, (ii) insert 22, (iii) insert 24, (iv) insert 7, (v) insert 9, (vi) delete 24. **(15 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q5. Write down the properties of an AVL tree, and prove that it is a balanced binary search tree. **(12 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q6. Suppose you are inserting a node in an AVL tree, and x is the lowest node violating AVL balance and x is left-heavy. Demonstrate the balancing procedure using figures. **(12 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q7. Insert a node with key = 20 to the given AVL tree. Then perform another insertion of a node with key = 68 on the resulting AVL tree. Show all the steps and the resulting AVL trees.



(10 marks) [CSE 207 | L-2/T-2 | 2020–21]

Q8. Prove or disprove:

(a) In an AVL tree, the difference in length between the shortest and the longest paths from the root to the leaves is at most one.

(b) The minimum number of nodes in an AVL tree with height 5 is 20.

(12 marks) [CSE 207 | L-2/T-2 | 2019–20]

Q9. What are the rotations that are required to balance an AVL tree? **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q10. How does a rotation balance an AVL tree? **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q11. What is the maximum height of an AVL tree having 10 nodes? **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q12. Prove that a tree with n nodes where the heights of the two children of any node differ by at most 1 has $O(\log n)$ height. **(12 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q13. Mention one advantage of AVL trees over red-black trees, and one advantage of red-black trees over AVL trees. **(4 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q14. If the i-th and $(i+2)$-th Fibonacci numbers are x and y respectively (where $i > 0$), what is the minimum number of nodes in an AVL tree of height $(i-1)$? **(4 marks)** [CSE 207 | L-2/T-2 | 2017–18]

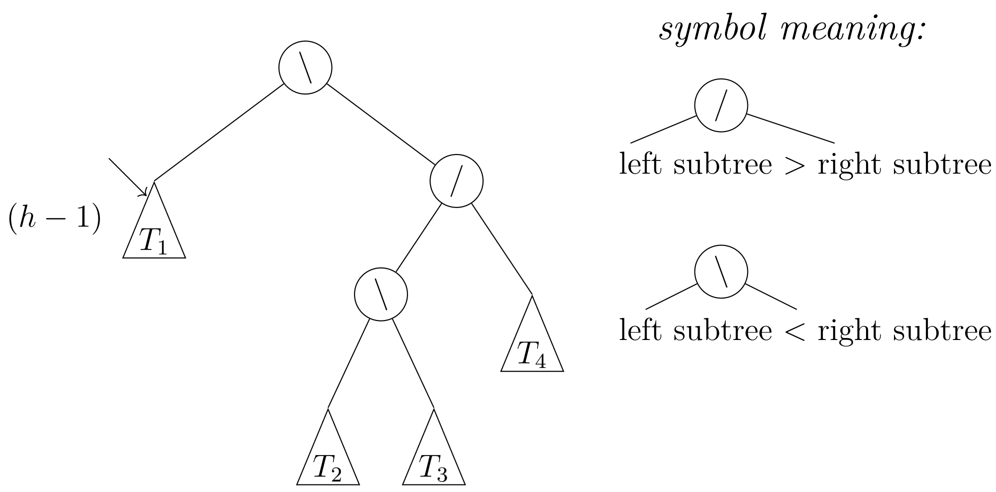
Q15. What is an AVL tree? Draw the AVL tree that results after successively inserting the keys $11, 4, 15, 25, 36, 47, 61, 53, 75$ into an initially empty AVL tree. **(10 marks)** [CSE 203 | L-2/T-1 | 2015–16]

Q16. Explain, with examples, the single rotation and double rotation operations on AVL trees. **(10 marks)** [CSE 203 | L-2/T-1 | 2014–15]

Q17. What is a Multiway Search tree? How do we search a key in a Multiway Search tree? **(10 marks)** [CSE 203 | L-2/T-1 | 2014–15]

Q18. Write the properties of a B-tree. **(5 marks)** [CSE 203 | L-2/T-1 | 2013–14]

Q19. Let T be an AVL tree where T_1 has height $h-1$. Determine the heights of T_2, T_3, and T_4. Discuss what happens when you delete an element from T_1; cover all possible cases for deletion in an AVL tree. **(4+4+7=15 marks)** [CSE 203 | L-2/T-1 | 2012–13]



Q20. Design an AVL tree by inserting the keys $5, 12, 3, -2, -5, -3, 1$ in order. Show all steps. **(15 marks)** [CSE 203 | L-2/T-1 | 2012–13]

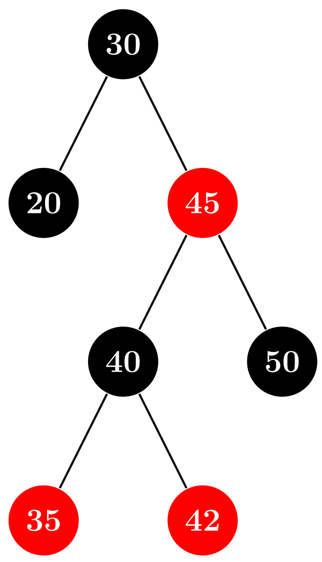
Q21. What is the height of an AVL tree with n internal nodes? **(5 marks)** [CSE 203 | L-2/T-1 | 2012–13]

Q22. Build a 2-3-4 tree with the keys $12, 5, 6, 10, 15, 8, 13, 17, 11, 14, 4$. Then delete elements in the following sequence: $4, 12, 13$. Show all steps. **(25 marks)** [CSE 203 | L-2/T-1 | 2012–13]

Red-Black Trees

Q1. Starting from an empty red-black tree, perform the insertion operation of the keys 41, 38, 31, 12, 19, 8, 25, and 32 in the given order while maintaining all red-black properties. After completing all insertions, use the resulting tree and perform deletions of the keys 8, 12, and 19 in that order. During the insertion or deletion, you must show the tree after each operation, including the color of every node. **(25 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q2. Delete the node 50 from the given Red-Black Tree.

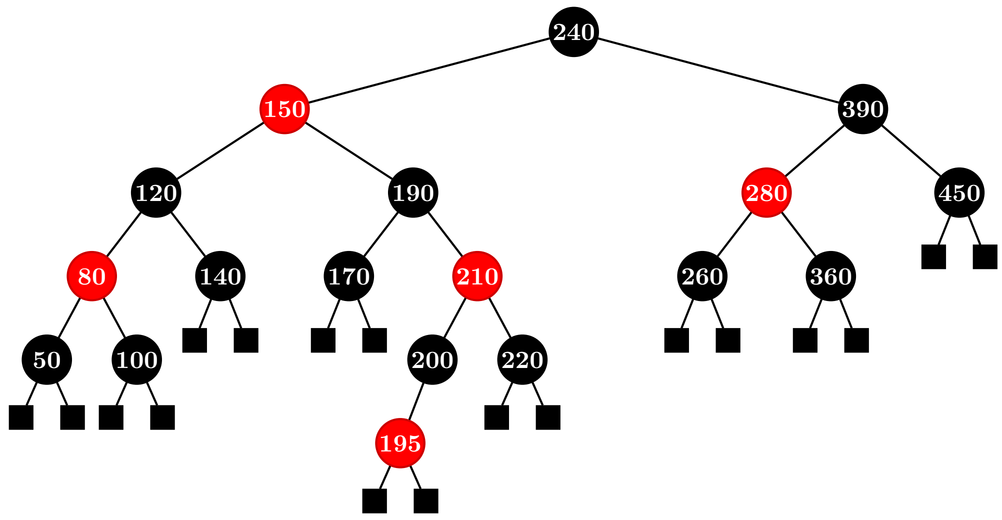


(10 marks) [CSE 207 | L-2/T-1 | 2023–24]

Q3. Write the pseudo-code of an insertion operation in a Red-Black tree and explain (graphically) different forms of transformation necessary to maintain the RB property while inserting an item. Briefly compare RB-tree and AVL-tree. **(15 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q4. What are the properties of a red-black tree? Derive that if a tree with n keys follows the red-black properties, its height h can be bounded as $h \le 2\lg(n+1)$. **(15 marks)** [CSE 207 | L-2/T-2 | 2021–22]

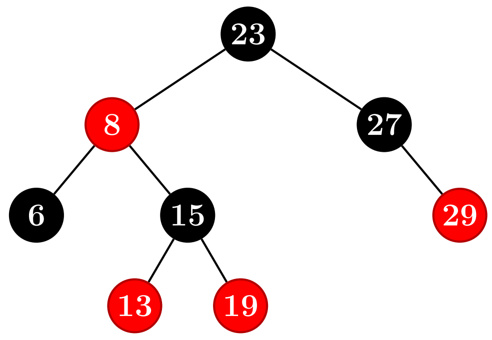
Q5. Usually we have to fix a Red-Black tree after deleting a node when the properties of the Red-Black tree get violated because of node deletion. Explain whether it is possible to do so without performing any rotation. Delete the node with key $= 150$ from the given Red-Black tree. Show all the steps and the resulting Red-Black tree.



(20 marks) [CSE 207 | L-2/T-2 | 2020–21]

Q6. Mr. X has proposed a new algorithm for insertion to a Red-Black tree, where the color of the newly inserted node is set to black by default. Discuss the advantages and disadvantages of this algorithm over the traditional algorithm followed for insertion to a Red-Black tree. **(5 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q7. Insert 11 into the Red-Black tree shown in the figure. You have to show the intermediate steps.



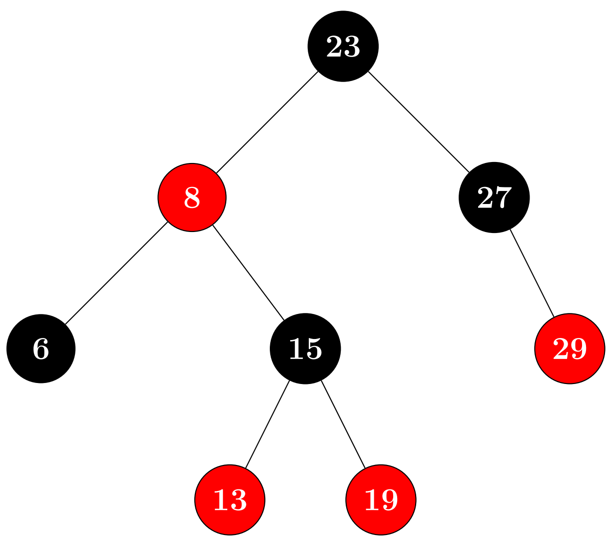
(15 marks) [CSE 207 | L-2/T-2 | 2019–20]

Q8. When we insert a node into a red-black tree, we initially set the color of the new node to red. Why don't we choose to set the color to black? **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q9. Would inserting a new node to a red-black tree and then immediately deleting it change the tree? Explain with an example. **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q10. What is the ratio between the lengths of the longest path and the shortest path in a red-black tree? **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q11. Insert 11 into the given Red-Black tree, showing each operation. **(15 marks)** [CSE 207 | L-2/T-2 | 2017–18]



Q12. Write the properties of a red-black tree. Show that the height of a red-black tree T storing n items is $O(\log n)$. Explain, with examples, the three cases that may arise for inserting a node in a red-black tree. **(15 marks)** [CSE 203 | L-2/T-1 | 2015–16, 2014–15]

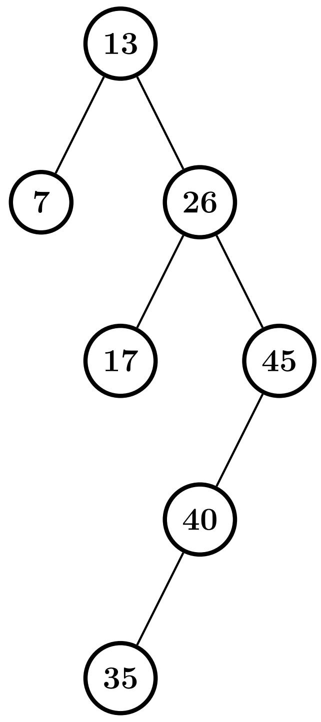
Q13. Explain, with examples, the four cases that may arise for deleting a node from a red-black tree. **(15 marks)** [CSE 203 | L-2/T-1 | 2014–15]

Q14. Write the properties of a red-black tree. Show the red-black trees that result after successively inserting the keys $51, 46, 41, 22, 29, 18$ into an initially empty red-black tree. **(15 marks)** [CSE 203 | L-2/T-1 | 2013–14]

Q15. Write down the properties of a red-black tree. With an example, show the insertion and deletion operations and their costs in a red-black tree. **(5+15=20 marks)** [CSE 203 | L-2/T-1 | 2012–13]

Splay Trees

Q1. Complete the following operations on the given splay tree: (i) Insert 30, (ii) Delete 35. Show all the steps.



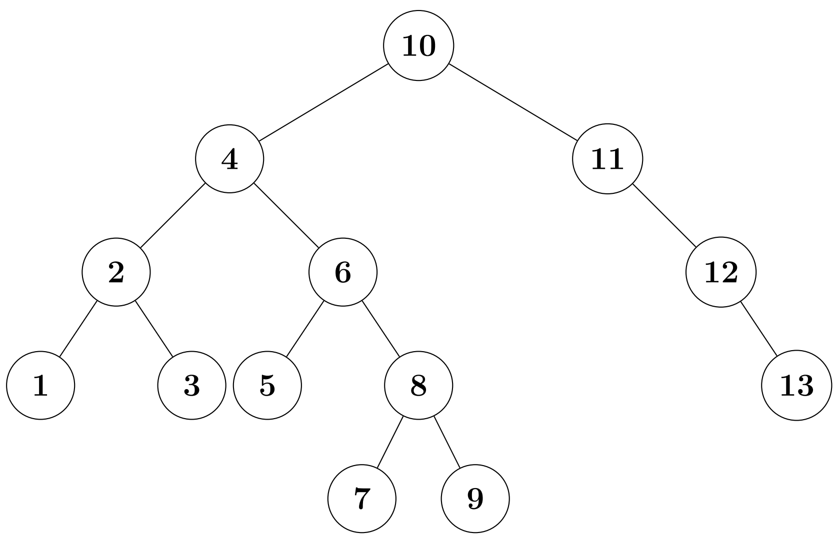
(13 marks) [CSE 207 | L-2/T-1 | 2023–24]

Q2. How is the concept of potential function used to design a splay tree? Explain. **(8 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q3. What are the splay-step operations to make a binary search tree a balanced one? **(8 marks)** [CSE 207 | L-2/T-2 | 2021–22]

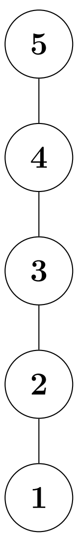
Q4. Define the rank of a node of a splay tree. Prove that the change in potential of a splay tree not resulting from any splay after an insertion operation is $O(\log n)$, where the splay tree had n nodes before the insertion operation. **(15 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q5. (i)For the splay tree of the following figure, show the resulting tree after successively accessing the keys 3 and 9 in the splay tree. (ii)For the splay tree of the following figure, show the resulting tree after deleting key 6 after doing operations in (i) (i.e., accessing the keys 3 and 9).



(15 marks) [CSE 207 | L-2/T-2 | 2018–19]

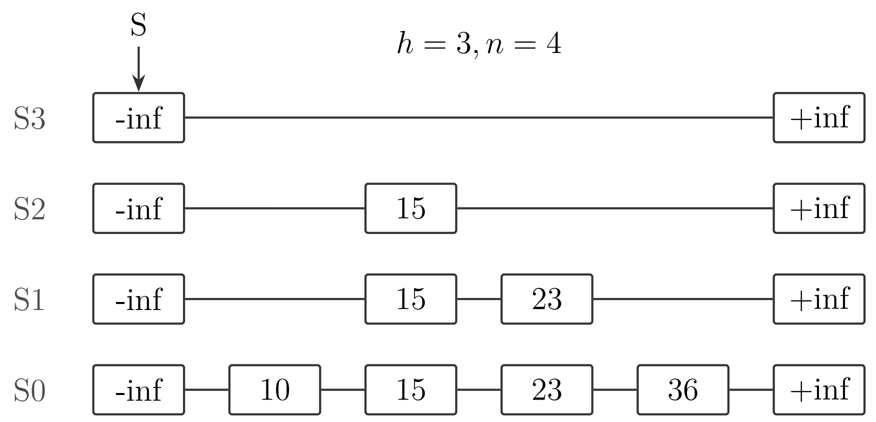
Q6. Perform SPLAY(1) on the given binary search tree, showing all intermediate steps and operations performed at each node. **(10 marks)** [CSE 207 | L-2/T-2 | 2017–18]



Q7. What is the main objective of splay trees? Do splay trees explicitly try to balance the height? **(5 marks)** **[CSE 207 | L-2/T-2 | 2017–18]**

Skip Lists

Q1. Prove that the expected space usage of a skip list having n items is $O(n)$. Perform the following operations on the given skip list sequentially: (i) Insert key = 17 where $i = 2$, (ii) Insert key = 25 where $i = 4$, (iii) Delete key = 25. Show all the steps and the resulting skip lists.



(15 marks) **[CSE 207 | L-2/T-2 | 2020–21]**

Q2. In a skip list with n entries,

(a) Prove that the expected space used is $O(n)$.

(b) Prove that the expected search and insertion time is $O(\log n)$.

(15 marks) **[CSE 207 | L-2/T-2 | 2018–19]**

Q3. Compare linked lists and skip lists. Show that the expected space used by a skip list with n entries is $O(n)$. Draw the skip-list for the following table that shows the keys and consecutive numbers of heads found in a coin toss while inserting the keys:

Key	15	40	35	28	21	65	84	79	50
No. of consecutive heads	1	4	1	0	3	2	2	3	1

(15 marks) **[CSE 203 | L-2/T-1 | 2014–15, 2013–14]**

Q4. Write algorithms for the search, insertion and deletion operations of skip lists. Show that the expected search, insertion and deletion time is $O(\log n)$ in a skip list with n entries. Also show that the expected space used is $O(n)$. **(18 marks)** **[CSE 203 | L-2/T-1 | 2013–14]**

Q5. Draw the skip-list for the following table that shows the keys and consecutive numbers of heads found in a coin toss while inserting the keys in a probabilistic skip-list:

Key	3	6	7	9	12	17	19	21	25	26
No. of consecutive heads	1	4	1	2	1	2	1	1	3	1

Perform the following operations on the skip-list and show the procedures:

(a) Search 21

(b) Insert 22 with 3 consecutive heads

(c) Delete 19

(d) Search 18

(7+8=15 marks) **[CSE 203 | L-2/T-1 | 2011–12]**

Q6. Show that the expected running time of searching in a probabilistic skip-list is $O(\log n)$. **(6 marks)** **[CSE 203 | L-2/T-1 | 2011–12]**

Advanced Heaps (Binomial Heaps)

Q1. Prove that there are $\binom{k}{i}$ nodes at depth i in a binomial heap B_k of order k. **(8 marks)** **[CSE 207 | L-2/T-1 | 2024–25, 2019–20, 2018–19]**

Q2. Let H_1 and H_2 be binomial heaps. Suppose the highest order tree in heap H_1 is of order m, and the highest order tree in H_2 has degree $m + 1$. Prove or disprove: "H_2 must have more nodes than H_1, regardless of how many trees are present in H_1 and H_2." **(7 marks)** **[CSE 207 | L-2/T-1 | 2024–25]**

Q3. Prove the following properties for a binomial tree B_k of order k:

(a) The number of nodes is 2^k.

(b) The height is k.

(c) The degree of root is k.

(15 marks) [CSE 207 | L-2/T-1 | 2023–24]

Q4. Perform union of two binomial min heaps with six and seven nodes respectively. You can use any valid values in the nodes. **(10 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q5. Answer the following questions for two max priority queues `pq1` and `pq2` implemented using binomial heap. Both are initially empty.

(a) Insert $-5, 20, 4, 19, 1, -3, 6, 12, 3, 2, 18$ in `pq1` and $25, 21, 10, 13, 17, 9, 8, 11, 7$ in `pq2`. Draw the two binomial heaps after the insertion.

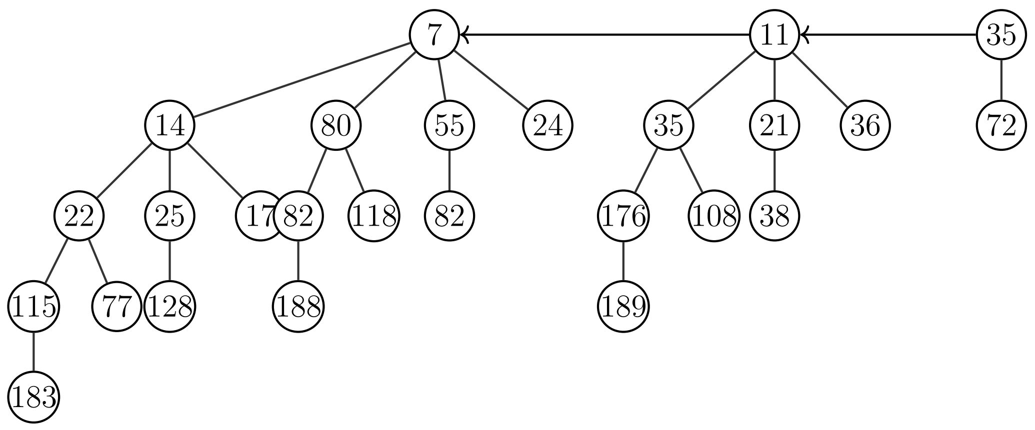
(b) Meld `pq1` and `pq2` and then decrease the key of node 20 to -20 from the melded priority queue. Show the necessary steps.

(c) What will be the total actual cost in credits if we call the Extract-Max operation two times consecutively on the resultant priority queue? Assume that unlinking a node from its children takes one credit, two node addition operations take one credit, an insertion of a node takes one credit, and a two-key comparison takes one credit. All other operations take zero credit.

(23 marks) [CSE 207 | L-2/T-1 | 2022–23]

Q6. Prove that in a binomial heap, the amortized cost of INSERT is $O(1)$ and the worst-case cost of EXTRACT-MIN and DECREASE-KEY is $O(\log n)$. **(12 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q7. Suppose there are two priority queues named pq1 and pq2. The pq1 is a max priority queue and pq2 is a min priority queue. Both the priority queues are implemented using the binomial heap data structure. The priority queue pq1 was initially empty, but then the following numbers were inserted in this order: 210, 201, 220, 202, 221, 205, and 209. The internal heap representation of the priority queue pq2 is show in figure. Now answer the following questions:



(a) Formulate an algorithm or pseudocode that will convert a min priority queue to a max priority queue. The amortized running time of the algorithm has to be $O(n)$ where n is the total length of the min priority queue. Note that the priority queues are implemented using the binomial heap data structure. Also, analyze the amortized running time of your proposed algorithm using the potential method.

(15 marks) [CSE 207 | L-2/T-2 | 2021–22]

(b) Apply your proposed algorithm on `pq2` to convert it to a max priority queue. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

(c) Apply the meld operation on `pq1` and `pq2` and store it to a new priority queue `pq3`. Show the internal heap representation of `pq3`. Note that you have to work with the updated `pq2`. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

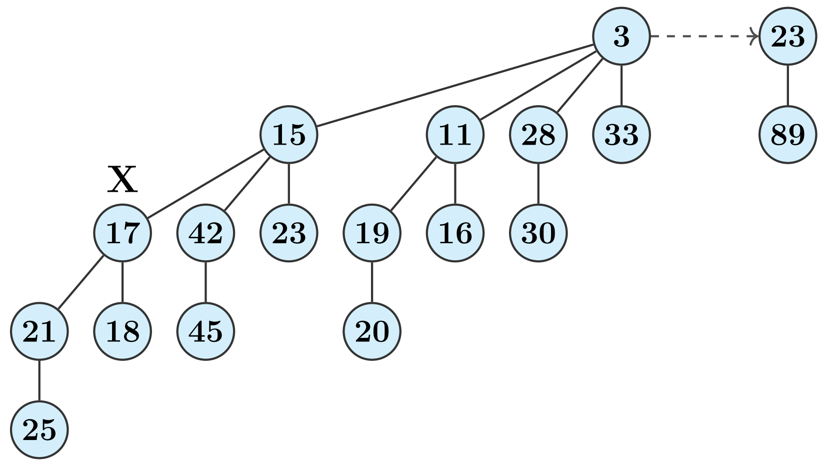
(d) Determine the internal heap representation of `pq3` after executing extract-max operation three times on `pq3`. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

(e) Evaluate the total actual cost in credits of the extract-max operations in the previous question. Assume that unlinking a node from its children takes one credit, a node addition operation takes one credit, an insertion of a node takes one credit, and a comparison takes one credit. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

(f) A friend of yours claims that it is possible to implement Find-Min operation in a Binomial Min Heap in $O(1)$ time using the idea of Fibonacci Min Heap. Justify his claim with proper explanation. **(10 marks)** [CSE 207 | L-2/T-2 | 2020–21]

(g) A binomial heap has 19 nodes. Find out the binomial trees this binomial heap has. **(5 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q8. Show the binomial heap that results when the node X with key 17 is deleted from the binomial heap shown in the Figure below. Show the intermediate steps.



(17 marks) [CSE 207 | L-2/T-2 | 2019–20]

Q9. Mention the amortized running times of the following operations for binomial and Fibonacci heaps:

(a) Union

(b) Find min

(c) Extract min

(d) Decrease key

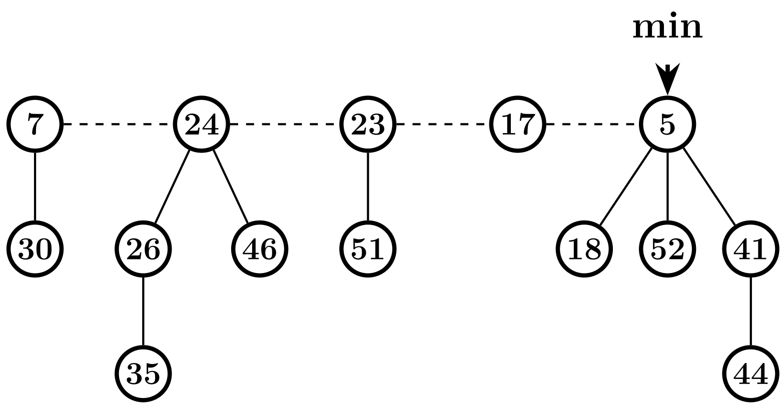
(8 marks) [CSE 207 | L-2/T-2 | 2019–20, 2018–19]

Q10. Write down the running times of the following operations for both Binomial heap and Fibonacci heap: (i) Insert key, (ii) Find min, (iii) Extract min, (iv) Union of two heaps, (v) Decrease key. A friend claims to have designed a mergeable heap with $O(1)$ time for all operations above. Would you believe it? Justify. **(6+6 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q11. Prove that there are $\binom{k}{i}$ nodes at depth i in a binomial tree B_k. **(5 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Advanced Heaps (Fibonacci Heaps)

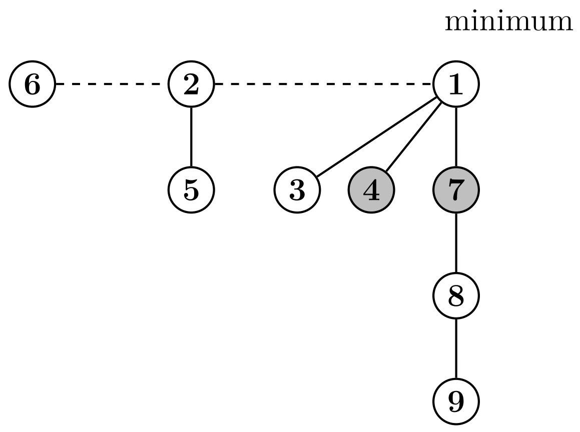
Q1. Show the Fibonacci heap that results when the minimum node (5) is deleted from the given Fibonacci heap. Show the intermediate steps.



(12 marks) [CSE 207 | L-2/T-1 | 2024–25]

Q2. Draw the maximally damaged tree resulting from B_6. **(8 marks)** [CSE 207 | L-2/T-1 | 2024–25, 2019–20]

Q3. Illustrate step by step what happens if the Extract-Min operation is executed on the given Fibonacci heap (nodes with keys 6, 2, 1, 5, 3, 4, 7, 8, 9 where 1 is the minimum).



(15 marks) [CSE 207 | L-2/T-1 | 2023–24]

Q4. Recall the deletion operation of the Fibonacci Heap data structure. Professor Rahsut has proposed a variant of the deletion operation (RAHSUT-DELETE). For this variant:

```
RAHSUT-DELETE(H, x)
    if x == H.min
        FIB-HEAP-EXTRACT-MIN(H)
    else
        y = x.p
        if y != NIL
            CUT(H, x, y) // cuts the link between x and its parent y, making x a root.
            CASCADING-CUT(H, y) // recursively marks and cuts the parents if has to
        add x's child to the root list of H
        remove x from the root list of H
```

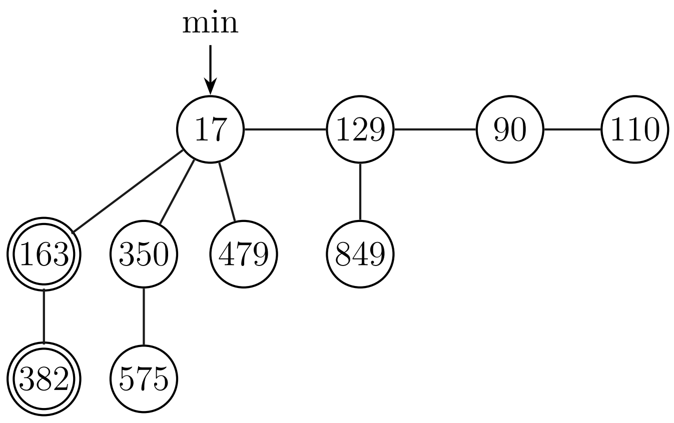
(a) Construct a good upper bound on the actual time of RAHSUT-DELETE when x is not `H.min`. Your bound should be in terms of `x.degree` and the number c of calls to the CASCADING-CUT procedure.

(b) Suppose we call RAHSUT-DELETE(H, x) and let H' be the Fibonacci heap that results. Assuming that node x is not a root, find the potential of H' in terms of `x.degree`, the number c of calls to the CASCADING-CUT procedure, $\Phi(H)$, and $m(H)$.

(12 marks) [CSE 207 | L-2/T-1 | 2022–23]

Q5. Describe how Fibonacci numbers are related to Fibonacci Heaps. **(5 marks)** [CSE 207 | L-2/T-2 | 2020–21]

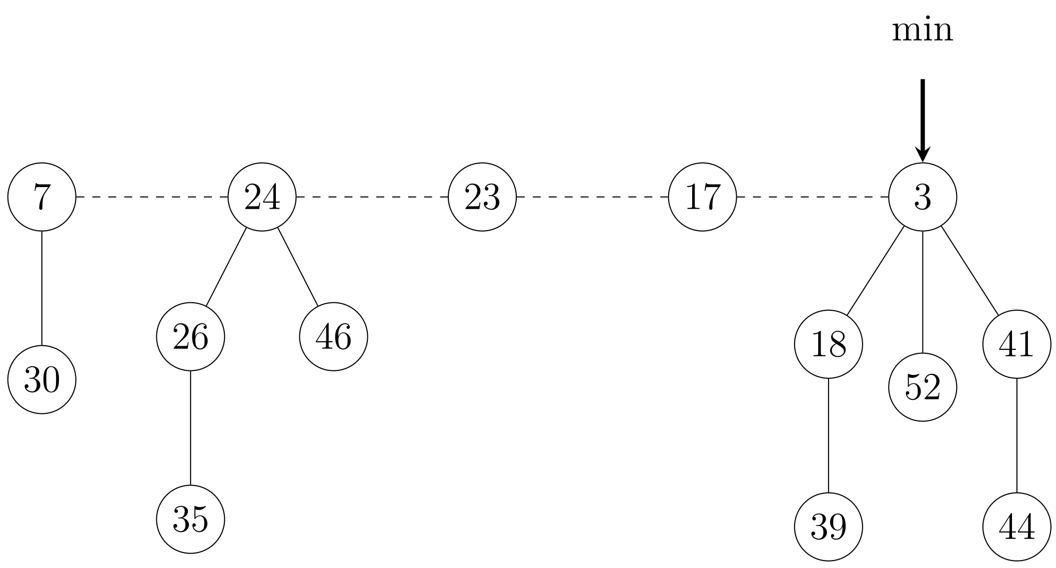
Q6. Prove or disprove the claim that "We do not mark the root node of any tree in the decrease key operation performed on a Fibonacci Min Heap, even if the root node loses a child". Perform an extract-min operation on the Fibonacci Min Heap shown below. Show all the steps and the resulting Fibonacci Min Heap.



Here double-circled nodes are marked and single-circled nodes are unmarked.

(15 marks) [CSE 207 | L-2/T-2 | 2020–21]

Q7. Show the Fibonacci heap that results when the minimum node (3) is deleted from the Fibonacci heap shown below. Show the intermediate steps.



(15 marks) [CSE 207 | L-2/T-2 | 2018–19]

Q8. Write the algorithm for extracting the minimum key from a Fibonacci heap. **(8 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q9. What is a maximally damaged tree in the context of Fibonacci heaps? Draw the maximally damaged tree of order 5 (degree of root is 5). **(4+6 marks)** [CSE 207 | L-2/T-2 | 2017–18]

BUET · CSE 207 · Data Structures & Algorithms II

Part III

Hashing

Hash Functions and Collision Resolution

S3 — Hashing

Hash Functions and Collision Resolution

Q1. Consider a hash table of size 11 into which the keys 10, 22, 31, 4, 15, 28, 17, 88, and 59 are inserted in the given order using $h(k) = k \bmod 11$. For collision resolution, four different strategies are to be used independently: (i) separate chaining, (ii) linear probing with $h(k, i) = (h(k) + 2i) \bmod 11$, (iii) quadratic probing with $h(k, i) = (h(k) + 3 + i^2) \bmod 11$, and (iv) double hashing where $h_1(k) = k \bmod 11$ and $h_2(k) = 7 - (k \bmod 7)$. For each method, construct the final hash table and compute the total number of unsuccessful probes required during insertion. **(25 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q2. Show that the expected time for a successful search in a hash table implemented using chaining is $O(1 + \alpha)$ where α is the load factor. **(8 marks)** [CSE 207 | L-2/T-1 | 2023–24, 2019–20, 2016–17, 2012–13]

Q3. Explain how you can handle collisions in a hash table using the double hashing method. **(7 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q4. Suppose you implemented a hashtable ADT that supports constant time insertion, deletion, and search. You used the hash function h1. On the contrary, your friend implemented another hashtable ADT using the hash function h2. The h1 hashes the keys 100, 130, 110, 150, 160, 200, 700, 800, 300 into 0, 3, 2, 0, 3, 1, 2, 5, and 8 respectively. The h2 hashes the same keys into 1, 4, 7, 10, 13, 4, 7, 10, 13. You made the hashtable size 16 but your friend made it to 17. For collision resolution, you used the quadratic probing function $f(i) = 2 \times i^2 + 3$. But your friend used the double- hashing technique and the second hash function was $h_2(x) = 19 - (x\%17)$. You inserted all the keys in the same order mentioned before to the hashtable. Then you deleted the key 100 and then searched for the key 150. But your hashtable implementation was showing "key 150 not found". Now, answer the following questions:

(a) Compare your implementation with your friend's by analyzing the total unsuccessful probes during insertion in both cases.

(b) Identify the possible core reason for not finding the key 150 during the search and provide a solution.

(20 marks) [CSE 207 | L-2/T-1 | 2022–23]

Q5. Suppose you implemented a hashtable ADT that supports constant time insertion, deletion, and search. You used the hash function $h1$. On the contrary, your friend suggested using the hash function $h2$. The $h1$ hashes the keys 100, 130, 110, 150, 160, 200, 700, 800, 300 into 0, 3, 2, 0, 3, 1, 2, 5, and 8, respectively. The $h2$ hashes the same keys into 1, 4, 7, 10, 13, 4, 7, 10, 13. You made the hashtable size 15 but your friend suggested making it 17. You used the quadratic probing function $f(i) = 3 \cdot i^2 + 5$ for collision resolution. Your friend suggested using the double-hashing technique. The second hash function is $h_2(x) = 11 - (x\%17)$. You inserted all the keys in the order mentioned before to the hashtable. Then you deleted the key 110 and then searched for the key 700. But your hashtable implementation was showing "key 700 not found". Now, answer the following questions:

(a) **[Comparing hash functions]** Compare two hash function implementations with quadratic probing vs. double hashing by analyzing the total unsuccessful probes during insertion in both cases.
(15 marks) [CSE 207 | L-2/T-2 | 2021–22]

(b) Comment on a friend's suggestion regarding the hash table size. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

(c) Identify the possible core reason for not finding a key during the search and provide a solution. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q6. Analyze the running time of a chaining hash table in both successful search and unsuccessful search cases. **(10 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q7. You are given the task to implement the hash table data structure for a new programming language. You may use either separate chaining or open addressing. In open addressing, there are various options for probing. Explain your choice precisely with proper justification. **(10 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q8. Suppose you have invited N people to a dinner, and you asked someone to enter their arrival times into a hash table. Assume that exactly one guest can enter the dinner hall at a particular time. Assuming separate chaining, briefly describe how you can achieve $1 + \log \alpha$ expected time for an unsuccessful search, where α is the load factor. **(10 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Q9. Consider successively inserting the keys 10, 22, 31, 4, 15, 28, 16, 37 and 59 into a hash table of size $m = 11$ with hash function $h(k) = k \bmod m$. Illustrate the resulting hash tables after inserting these keys using (i) linear probing, and (ii) quadratic probing. You do not need to show the intermediate steps, just show the hash table after inserting all the elements. **(15 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Q10. Assuming simple uniform hashing, prove that an unsuccessful search in separate chaining takes expected $O(1 + \alpha)$ work, where α is the load factor. **(12 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q11. Quadratic probing does not necessarily probe all locations in hash tables. Do you agree with this statement? Justify your answer. **(6 marks)** [CSE 207 | L-2/T-2 | 2018–19, 2017–18]

Q12. Consider successively inserting the keys 14, 3, 29, 3, 19, 16, 35, and 2 into a hash table of size $m = 11$ with hash function $h(k) = k \bmod m$. Illustrate the resulting hash table after inserting these keys using quadratic probing. You do not need to show the intermediate steps, just show the hash table after inserting all the elements.

(12 marks) [CSE 207 | L-2/T-2 | 2018–19, 2016–17]

Q13. Under simple uniform hashing, what is the expected number of items that may collide with an element x in a hash table with separate chaining, given m chains and n items? **(10 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q14. Prove that the first $\lceil m/2 \rceil$ probes in quadratic probing are distinct given m is a prime number. What is the major advantage of double hashing over quadratic probing? **(8+3 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q15. Explain 'Chaining' and 'Open Addressing' methods for solving the problem of collisions in a hash table. Explain the three commonly used techniques to compute the probe sequences required for open addressing. **(15 marks)** [CSE 203 | L-2/T-1 | 2015–16]

Q16. What makes a good hash function? Explain the three commonly used techniques to compute the probe sequences required for open addressing in a hash table. **(15 marks)** [CSE 203 | L-2/T-1 | 2014–15, 2011–12]

Q17. Draw the 11-item hash table that results from using the hash function $h(i) = (2i + 5) \bmod 11$ to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, 5, assuming collisions are handled by double hashing using a secondary hash function $h'(k) = 7 - (k \bmod 7)$. **(12 marks)** [CSE 203 | L-2/T-1 | 2013–14]

Q18. Write pseudocode for finding a key, inserting a key, and deleting a key in a hash table using quadratic probing. Explain your method with an example. **(12 marks)** [CSE 203 | L-2/T-1 | 2012–13, 2011–12]

Q19. Insert the keys $18, 41, 22, 44, 59, 32, 31, 73$ into a hash table of size 13 using double hashing with

$$h_1(k) = k \bmod 13, \qquad h_2(k) = 8 - (k \bmod 8).$$

Draw the resulting hash table. **(13 marks)** [CSE 203 | L-2/T-1 | 2012–13]

BUET · CSE 207 · Data Structures & Algorithms II

Part IV

NP-Completeness

P vs NP Classes, Reductions, NP-hard and NP-complete Problems

S4 — NP-Completeness

P vs NP Classes

Q1. Prove that $P = NP \Rightarrow NP = \text{co-}NP$. **(12 marks)** [CSE 207 | L-2/T-1 | 2024–25, 2018–19]

Q2. Prove or disprove that $P = NP \cap \text{co-}NP$. **(8 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q3. "It is possible that $P \neq NP$, yet $NP = \text{co-}NP$." Do you agree with this statement? Justify your answer. **(8 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q4. Define the classes of problems P, NP, and NP-complete. Draw a Venn diagram showing the widely believed relationship among these classes. **(9 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q5. Differentiate among NP, NP-hard, and NP-complete classes of problems. **(6 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q6. Using an example, show that if a problem is in NP, the complement of that problem is in co-NP. Draw the general consensus regarding P, NP and co-NP in a Venn diagram. **(10 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q7. Suppose $X \in$ NP-complete. Prove that $X \in P$ if and only if $P = NP$. **(8 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Q8. How do we cope with hard problems? Discuss. **(10 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q9. Suppose $X \in$ NP-complete. Prove that $X \in P$ if and only if $P = NP$. **(10 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q10. Prove that NP $\neq$ co-NP $\Rightarrow P \neq NP$. **(10 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q11. If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, show that $L_2 \in P$ implies $L_1 \in P$. If an NP-complete problem is polynomially solvable, prove that $P = NP$. **(10 marks)** [CSE 207 | L-2/T-2 | 2013–14]

Q12. Define the following six classes of problems: P, Co-P, NP, Co-NP, NPC, and NP-hard. By a diagram show the relationship among P, Co-P, NP, Co-NP, and NPC that most researchers regard as the most likely. **(6+4=10 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Reductions

Q1. For a graph $G = (V, E)$, a set of vertices I in G is an *independent set* if for every two vertices in I, there is no edge between them, and a set C of vertices in V is a *clique* if every two distinct vertices are adjacent. Given a graph G and a number K, the independent set problem is known to be NP-complete. The clique problem asks: given a graph $G = (V, E)$ and an integer K, does G have a clique of size at least K? Prove that the clique problem is as hard as the independent set problem. **(15 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Q2. In the *Set Cover* problem (decision version), we are given a set of elements U (the universe), a collection of subsets $S = \{S_1, S_2, \ldots S_m\}$ where each $S_i \subseteq U$, and an integer k. The goal is to determine whether there exists a sub-collection $C \subseteq S$ of size $\leq k$ such that $\bigcup_{S_i \in C} S_i = U$. Prove that the set cover problem is NP-complete.
Hint: Given a graph $G = (V, E)$ and an integer k, the vertex cover problem (decision version) is to determine whether there is a subset of vertices $V' \subseteq V$ with $|V'| \leq k$ such that every edge in E is incident to at least one vertex in V'. This problem is known to be NP-complete. **(15 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q3. Imagine you are managing a team of employees, each with their own set of tasks to complete. However, some tasks cannot be worked on simultaneously due to dependencies or resource constraints. Your goal is to maximize productivity by scheduling as many tasks as possible while ensuring that no two conflicting tasks are assigned to the same team member simultaneously. You described the problem to your friend. He commented, "The problem can be mapped to a popular NP-complete problem and can be proved to be NP-hard by reduction technique."
Justify your friend's comment by mapping the problem to an NP-complete problem and reducing it to another known NP-complete problem. **(17 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q4. Suppose you are a developer of a social media platform. The platform wants to add a feature in the admin section. The feature is to identify the minimal subset of users who can reach out to all other users of the platform through their (selected subset users') connections. Now you have to formulate an algorithm to develop the feature. Your colleague commented that the problem can be mapped to a popular np- complete problem.

(a) **[Minimal subset reachability problem]** Map the problem to the NP-complete problem that your colleague suggested. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

(b) **[Minimal subset reachability problem]** Prove that the problem is NP-complete. You have to use the 3-SAT problem in the proof. **(10 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q5. Explain how we can prove that an optimization problem is NP-complete. **(5 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q6. For a graph $G = (V, E)$, a set S of vertices in G is a *vertex cover* if every edge e in G has at least one end in S. Given a graph G and a number K, the vertex cover problem asks if G contains a vertex cover of size at most K. Given a set U of n elements, a collection $S_1, S_2, \ldots, S_m$ of subsets of U, and a number k, the set cover problem asks if there exists a collection of at most k of these subsets whose union is equal to U. Prove that the set cover problem is at least as hard as the vertex cover problem. **(12 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Q7. The TAUTOLOGY problem asks if a given Boolean formula is true for all possible assignments to the Boolean variables. Prove or disprove that TAUTOLOGY is in co-NP. **(6 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q8. For a graph $G = (V, E)$, a set S of vertices in G is a *vertex cover* if every edge e in G has at least one end in S. A set of vertices I in G is an *independent set* if for every two vertices in I, there is no edge between them. Given a graph G and a number K, the vertex cover problem asks if G contains a vertex cover of size at most K. Given a graph G and a number K, the independent set problem asks if G contains an independent set of size at least K. Prove that the vertex cover problem is at least as hard as the independent set problem. **(12 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q9. What do you mean by $X \leq_p Y$? Show that the vertex cover problem $\leq_p$ set cover problem. **(3+7 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q10. Given clauses $C_1, C_2, \ldots, C_k$ each of length 3 over Boolean variables (3-SAT), and a graph G with number K (Vertex Cover asks if G has a vertex cover of size $\leq K$). Prove that Vertex Cover is at least as hard as 3-SAT. **(15 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q11. Explain the term "reducibility". If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, show that $L_2 \in P$ implies $L_1 \in P$. If any NP-complete problem is polynomial-time solvable, prove that $P = NP$. **(10 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Q12. Define the classes P, NP, and NP-complete. What are the steps to prove a problem X is NP-complete? Prove that the decision version of the independent set problem is NP-complete using a reduction from 3-CNF-SAT. Give an intuitive sketch of the proof of the Cook-Levin theorem (CIRCUIT-SAT is NP-complete). **(4+4+12+5 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q13. If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, then show that $L_2 \in P$ implies $L_1 \in P$. If any NP-complete problem is polynomial-time solvable, then prove that $P = NP$. **(7+3=10 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Q14. **Instance of Satisfiability Problem**
Boolean variables: X_1, X_2, X_3, X_4, X_5
Clauses:

$$C_1 = X_1 \vee \overline{X_2}, \quad C_2 = \overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5}$$
$$C_3 = \overline{X_2}, \quad C_4 = X_2 \vee X_3 \vee \overline{X_4}$$

Mention the number of boolean variables and clauses in the 3-SAT problem.

NP-hard and NP-complete Problems

Q1. What are some approaches to deal with an NP-complete problem? **(6 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q2. Define the satisfiability problem. Show that the independent set problem is NP-hard by reducing the 3-SAT problem to the independent set problem. **(3+7 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q3. Your friend claims to have developed a polynomial-time algorithm to solve the set-packing problem. What will happen if this claim is proved to be true? **(5 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Q4. Write the names of four NP-complete problems. Define the following six classes of problems: P, Co-P, NP, Co-NP, NP-complete, and NP-hard. By a diagram show the relationship among these classes that most researchers regard as most likely. **(10 marks)** [CSE 207 | L-2/T-2 | 2013–14]

Q5. Show that the decision version of the vertex cover optimization problem is NP-complete using a reduction from the Independent Set decision problem. **(12 marks)** [CSE 207 | L-2/T-2 | 2012–13]

Q6. Prove that the optimization version of the Independent Set problem is NP-hard. **(8 marks)** [CSE 207 | L-2/T-2 | 2012–13]

Q7. What are the traveling-salesman problem, Euler tour problem and Hamiltonian cycle problem? If $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$, then prove that $L_1 \leq_p L_3$. **(3+7=10 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Q8. Prove that the decision version of the maximum clique problem is NP-complete by reducing from 3-CNF-SAT. **(17 marks)** [CSE 207 | L-2/T-2 | 2009–10]

Q9. What do you mean by NP-hard and NP-complete problems? Prove that finding a clique in a graph is NP-complete. **(5+12 marks)** [CSE 207 | L-2/T-2 | 2008–09]

Q10. Write down the procedure of proving that a problem is NP-complete. **(5 marks)** [CSE 207 | L-2/T-2 | 2008–09]

Q11. There are clauses $C_1 = (X_1 \vee \overline{X_2})$, $C_2 = (\overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5})$, $C_3 = \overline{X_2}$, and $C_4 = (X_2 \vee X_3 \vee \overline{X_4})$. How the above instance of the satisfiability problem can be transformed into an instance of 3-SAT problem? (X_1, X_2, X_3, X_4 and X_5 are boolean variables) **(13 marks)** [CSE 207 | L-2/T-2 | 2008–09]

Q12. Define: (i) size of a problem, (ii) time complexity function, (iii) the class P, (iv) the class NP, (v) NP-Complete problems. Show how an instance of the satisfiability problem can be transformed into an instance of 3-SAT. Prove that 3-colouring of a graph is NP-complete. **(10+10+15 marks)** [CSE 207 | L-2/T-2 | 2007–08]

Q13. Prove that 3-colouring of a graph is NP-complete. [CSE 207 | L-2/T-2 | 2007–08]

Q14. What is the difference between NP-complete and NP-hard problems?
Prove that the satisfiability of Boolean formulas is NP-complete.
Prove that 3-SAT $\leq_p$ Independent Set.
Give an example of reduction by equivalence . **(5+15+10 marks)** [CSE 207 | L-2/T-2 | 2006–07]

BUET · CSE 207 · Data Structures & Algorithms II

Part V

Coping with Hardness

Approximation Algorithms, Backtracking, Branch and Bound

S5 — Coping with Hardness

Approximation Algorithms

Q1. Give an approximation algorithm for the vertex cover problem. Analyze the approximation ratio of your algorithm. **(15 marks)** [CSE 207 | L-2/T-1 | 2024–25, 2018–19, 2016–17]

Q2. Let $\pi = \pi_1\pi_2\dots\pi_n$ be a permutation of $1\ 2 \dots n$ (the identity permutation). A reversal (i, j) reverses the contiguous subsequence between i and j i.e. transforms $\pi_1 \dots \pi_{i-1}\pi_i\ \pi_{i+1} \dots \pi_{j-1}\ \pi_j\ \pi_{j+1} \dots \pi_n$ into $\pi_1 \dots \pi_{i-1}\pi_j\ \pi_{j-1} \dots \pi_{i+1}\ \pi_i\ \pi_{j+1} \dots \pi_n$. The sorting by reversals problem is to find the minimum number of reversals that transforms a given permutation to the identity permutation. For example, the permutation [4 1 2 3] can be sorted by reversing [1 2 3] to get [4 3 2 1] and then reversing this entire sequence.

Let's extend the permutation by adding $\pi_0 = 0$ and $\pi_{n+1} = n + 1$ to the ends (π_0 and π_{n+1} are not moved during sorting). We say there is a breakpoint between π_i and π_{i+1} if they are not consecutive. For example, [4 1 2 3] is converted to [0 4 1 2 3 5] and there are three breakpoints: between 0 and 4, 4 and 1, and 3 and 5.

(a) Provide a lower bound on the number of reversals needed to sort a permutation π, in terms of the number of breakpoints in it.

(b) Design an approximation algorithm for the sorting by reversals problem with an approximation ratio at most 4. (Hint: consider maximal runs of contiguously increasing and contiguously decreasing elements.)

(15 marks) [CSE 207 | L-2/T-1 | 2023–24]

Q3. Give a 2-approximation algorithm for the vertex cover problem and prove that the algorithm computes the vertex cover at most twice the size of the actual vertex cover size. **(15 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q4. Suppose you want to fill up your luggage with the required items for your upcoming tour. But the luggage has a capacity of 600 kg. The items have different values and weights which are given below:

Item	Value	Weight
1	1200	150
2	1100	100
3	950	250
4	1350	350
5	800	200
6	1500	300

You want to fill up your luggage so that the total values of the items are maximized. So you plan to use an approximation algorithm that you learned from your teacher to solve the problem. The approximation algorithm scales down the high values by $\epsilon V_{max}/2n$ (ϵ = precision parameter, V_{max} = largest value and n = total items) and utilizes a dynamic programing technique. The dynamic programing technique uses the subproblem $OPT(i, v)$ (the minimum weight of the luggage for which we can obtain a solution of value at least v using a subset of items 1 to i), instead of using $OPT(i, w)$ (the max value subset of items 1 to i with weight limit w).

(a) **[Luggage maximization using scaling and dynamic programming]** Why does the approximation algorithm use the $OPT(i, v)$ subproblem instead of $OPT(i, w)$? **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

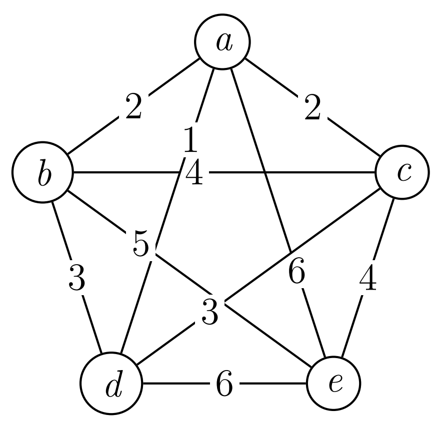
(b) Apply the approximation algorithm to solve the luggage problem with $\varepsilon = 0.1$. **(15 marks)** [CSE 207 | L-2/T-2 | 2021–22]

(c) Prove that for any $\varepsilon > 0$, the approximation algorithm computes a feasible solution whose value is within a $(1+\varepsilon)$ factor of the optimum in $O(n^3/\varepsilon)$. **(15 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q5. In the bottleneck traveling salesperson problem (TSP), we are given an undirected graph G with weights on its edges and asked to find a tour that visits the vertices of G exactly once and returns to the start so as to minimize the cost of the maximum-weight edge in the tour. Assuming that the weights in G satisfy the triangle inequality, design a polynomial-time 3-approximation algorithm for bottleneck TSP. **(13 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q6. What is fixed-parameter tractability? Why is it useful? **(7 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q7. Show how to construct a near optimal solution using a minimum spanning tree for the symmetric traveling salesman problem with distances that satisfy the triangle inequality.



(12 marks) [CSE 207 | L-2/T-2 | 2019–20]

Q8. Discuss the approximability of the general Traveling Salesperson Problem (TSP). Give an approximation algorithm for the Euclidean TSP problem and compute the approximation ratio of your algorithm. (2+3+5 marks) [CSE 207 | L-2/T-2 | 2017–18]

Q9. Give an approximation algorithm for the set cover problem and compute the approximation ratio. (3+7 marks) [CSE 207 | L-2/T-2 | 2017–18]

Q10. What is the vertex cover problem? Write a polynomial-time approximation algorithm for it. Prove its time complexity and find the approximation ratio. For the general traveling-salesman problem, prove that one cannot find good approximate tours in polynomial time unless $P = NP$. (7+8=15 marks) [CSE 207 | L-2/T-2 | 2015–16]

Q11. Consider the following instance of the maximum satisfiability (MAX-SAT) problem.

Set of Boolean variables, $V = \{w, x, y, z\}$

Set of clauses, $C = \{C1, C2, C3, C4, C5, C6\}$

where $C1 = (w \vee \bar{x} \vee \bar{y} \vee z)$, $C2 = (w \vee \bar{y})$, $C3 = (x \vee \bar{y})$, $C4 = (y \vee \bar{z})$

$C5 = (z \vee \bar{w})$, $C6 = (\bar{w} \vee \bar{x})$

Now find an assignment of variables such that maximum number of clauses is satisfied. Compute your solution using branch and bound algorithm. Write down a suitable bound function for the problem. Show detailed calculation steps through a branch and bound search tree.(12 marks) [CSE 207 | L-2/T-2 | 2014–15]

Q12. Write a polynomial-time 2-approximation algorithm for the traveling-salesman problem where the edge weights satisfy the triangle inequality, and prove the approximation ratio. If $P = NP$, prove that for any constant $\rho > 1$, there is no polynomial-time approximation algorithm with ratio ρ for the general TSP. (15 marks) [CSE 207 | L-2/T-2 | 2013–14]

Q13. Consider the following greedy approximation algorithm for the vertex cover optimization problem.

```
GreedyVertexCover(G):
    C = {}
    while E is not empty:            // E is the set of edges
        u = vertex in V with maximum degree   // V is the set of vertices
        V = V - {u}
        C = C union {u}
        for each edge (u, v) adjacent to u:
            E = E - {(u, v)}         // remove all edges covered by vertex u
    return C                         // C is the vertex cover
```

(a) Give an example graph where the above algorithm may not produce the optimal vertex cover.

(b) Show that the approximation ratio of the above greedy algorithm is $O(\lceil \lg |E| \rceil)$.

(15 marks) [CSE 207 | L-2/T-2 | 2012–13]

Q14. Give an efficient 2-approximation algorithm for vertex cove: optimization problem. (8 marks) [CSE 207 | L-2/T-2 | 2012–13]

Q15. Write a polynomial-time 2-approximation algorithm for the vertex-cover problem, and prove the approximation ratio of the algorithm. If $P = NP$, then prove that for any constant $\rho \geq 1$, there is no polynomial-time approximation algorithm with ratio ρ for the general traveling-salesman problem. (8+7=15 marks) [CSE 207 | L-2/T-2 | 2011–12]

Q16. Give a 2-approximation algorithm for the Traveling Salesperson Problem (TSP) for graphs obeying the triangle inequality. Prove the approximation ratio. (17 marks) [CSE 207 | L-2/T-2 | 2009–10]

Q17. Write a 2-approximation algorithm for the "vertex cover" problem. Show that your algorithm has the mentioned approximation ratio. (10 marks) [CSE 207 | 2008–2009]

Q18. Write a 2-approximation algorithm for the vertex cover problem. Justify your answer. (10 marks) [CSE 207 | L-2/T-2 | 2006–07]

Backtracking

Q1. Consider the following Boolean formula:

$$(x' \vee y') \wedge (x' \vee y) \wedge (x \vee y' \vee z) \wedge (x \vee y \vee z)$$

Use a backtracking algorithm to decide whether this is satisfiable or not. Show the backtracking tree. (10 marks) [CSE 207 | L-2/T-1 | 2024–25]

Q2. Consider the following boolean formula:

$$(x \vee y') \wedge (x' \vee w) \wedge (x \vee y \vee z') \wedge (x' \vee w \vee z)$$

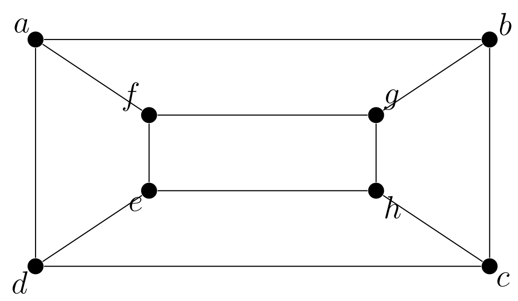
Use the backtracking algorithm to decide whether the boolean formula is satisfiable or not. If satisfiable, find all the assignments of the boolean variables for which the boolean formula is satisfiable. You must show the backtracking tree. **(10 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q3. Consider the following Boolean formula:

$$(x' \vee y') \wedge (x \vee y') \wedge (x' \vee y) \wedge (x \vee y \vee z') \wedge (x \vee y \vee z)$$

Use the backtracking algorithm to decide whether this is satisfiable or not. You must show the backtracking tree. **(10 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Q4. Apply backtracking to the problem of finding a Hamiltonian circuit in a given graph. Show the corresponding state-space tree. **(10 marks)** [CSE 207 | L-2/T-2 | 2017–18]



Q5. Discuss the Presumed Optimal Solution for the Multipeg Tower of Hanoi Problem with its properties. Solve the problem for $(n, p) = (395, 8)$. **(5+10 marks)** [CSE 203 | L-2/T-1 | 2017–18]

Q6. Consider the following Boolean formula: $(x' \vee y)(x \vee y' \vee z)(y \vee y')(x \vee y)(x \vee y \vee z')(x \vee y \vee z)$. Use a backtracking algorithm to decide whether it is satisfiable. Show the backtracking tree. **(15 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Q7. Compare the techniques of brute-force algorithms, heuristics, and approximation algorithms for coping with hard problems. Write a backtracking algorithm to color a map with no more than four colors. **(5+5=10 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Q8. Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (377, 8)$. Find $k_{\max}$, N_0, and n_1 for at least two subproblems. **(10 marks)** [CSE 207 | L-2/T-2 | 2013–14]

Q9. Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (270, 7)$, showing the values of k, max, N_0, n_1. Present the solution by drawing a binary tree. **(15 marks)** [CSE 207 | L-2/T-2 | 2008–09]

Q10. Discuss the Multi-peg Hanoi problem. Argue in favour of the recurrence relation to be satisfied by the presumed optimal solution. Prove the properties of the presumed optimal solution. **(5+5+10=20 marks)** [CSE 207 | L-2/T-2 | 2008–09]

Q11. Write down the backtracking technique for solving the 4-Queen problem. Give the state tree also. **(10+5=15 marks)** [CSE 207 | L-2/T-2 | 2008–09]

Q12. Discuss the Multi-Peg Tower of Hanoi problem. Argue in favour of the recurrence relation satisfied by the presumed optimal solution. Solve the problem with $(n, p) = (271, 7)$, showing the solution in a binary tree with at least 4 nodes. **(15+20 marks)** [CSE 207 | L-2/T-2 | 2007–08]

Q13. Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (273, 7)$. Show the subproblems arising up to the second level of recursion using a binary tree. **(15 marks)** [CSE 207 | L-2/T-2 | 2006–07]

Branch and Bound

Q1. Consider the following instance of the 0-1 knapsack problem. Solve this instance using the branch and bound algorithm. Show the branch-and-bound tree. Knapsack capacity = 10.

Item	Weight	Value
1	4	40
2	3	50
3	5	70
4	3	45

(15 marks) [CSE 207 | L-2/T-1 | 2024–25]

Q2. Explain branch-and-bound algorithms with an illustrative example. **(15 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Q3. You want to fill up your luggage with required items for an upcoming tour. The luggage has a capacity of 60 kg. The items have the following values and weights:

Item	Value	Weight (kg)
1	120	15
2	110	10
3	95	25
4	135	35
5	80	20
6	150	30

Solve this 0-1 knapsack problem using the branch and bound technique to maximize total value. **(15 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q4. There are 4 places in your village. You want to start from a place, then visit all other places and return to the first place without visiting the same place twice (other than the starting place), minimizing cost. The costs are given below:

From/To	1	2	3	4
1	0	10	15	20
2	5	0	12	18
3	8	9	0	17
4	11	14	16	0

Give an exponential time algorithm to solve this problem with running time $O(n^2 \cdot 2^n)$, and find the optimal tour using your algorithm. **(20 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Q5. There are 6 places in your village. You want to visit all the places and return to the first place that you will visit. Also, you don't want to visit the same place twice other than the starting place. You want to finish the visits with minimum cost. The cost of the roads between these places are given below:

From/To	1	2	3	4	5	6
1	0	10	15	20	25	30
2	5	0	12	18	24	29
3	8	9	0	17	22	28
4	11	14	16	0	21	27
5	13	19	23	26	0	31
6	28	27	25	24	22	0

Find the optimal tour using branch and bound, and pruning techniques combined. **(20 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Q6. In one approach to solve the Traveling Salesman Problem using branch-and-bound technique, the lower bound of the cost of completing a partial tour has been calculated from the cost of the minimum spanning tree spanning the remaining nodes of the graph. Decide whether this bound function will give correct answer all the time, with proper explanation. If there is any better bound function for this problem, mention it. **(5 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Q7. Solve the following instance of a 0-1 knapsack problem by a branch-and-bound algorithm using a state-space tree. You must show the branch-and-bound tree.

Knapsack capacity = 10.

Item	Weight	Value
1	2	10
2	3	9
3	2	20
4	5	30

(15 marks) [CSE 207 | L-2/T-2 | 2019–20]

Q8. Solve the following instance of the 0-1 knapsack problem using a branch-and-bound algorithm with a state-space tree (capacity = 10 kg):

Item	Weight (kg)	Profit (Taka)
1	3	18
2	5	35
3	4	44
4	7	36
5	2	18

(8 marks) [CSE 207 | L-2/T-2 | 2017–18]

Q9. Solve the following 0-1 knapsack instance (capacity = 10) using the branch-and-bound algorithm. Show the branch-and-bound tree.

Item	Weight	Value
1	2	10
2	3	9
3	2	20
4	5	30

(20 marks) [CSE 207 | L-2/T-2 | 2016–17]

Q10. Solve the following 0/1 knapsack instance (capacity = 15) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	140
2	6	132
3	4	140
4	3	90

Compare the backtracking and branch-and-bound techniques. **(3+12=15 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Q11. Consider the following instance of the maximum satisfiability (MAX-SAT) problem.

Set of Boolean variables, $V = \{w, x, y, z\}$

Set of clauses, $C = \{C1, C2, C3, C4, C5, C6\}$

where $C1 = (w \vee \bar{x} \vee \bar{y} \vee z)$, $C2 = (w \vee \bar{y})$, $C3 = (x \vee \bar{y})$, $C4 = (y \vee \bar{z})$

$C5 = (z \vee \bar{w})$, $C6 = (\bar{w} \vee \bar{x})$

Now find an assignment of variables such that maximum number of clauses is satisfied. Compute your solution using branch and bound algorithm. Write down a suitable bound function for the problem. Show detailed calculation steps through a branch and bound search tree.**(12 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Q12. What is a state-space tree? Solve the following instance of the 0/1 knapsack problem (capacity = 10) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	42
2	4	40
3	3	12
4	5	25

(15 marks) [CSE 207 | L-2/T-2 | 2013–14]

Q13. Applying the FIFO branch-and-bound technique on the 4-Queen problem, it is found that the technique is not efficient. Why? **(6 marks)** [CSE 207 | L-2/T-2 | 2008–09]

BUET · CSE 207 · Data Structures & Algorithms II

Part VI

String Matching Algorithms & Randomized Algorithms

Knuth-Morris-Pratt (KMP) Algorithm, Monte Carlo Algorithms & Probability Amplification, Las Vegas Algorithms & Randomized Quick Sort

S6 — String Matching Algorithms

Knuth-Morris-Pratt (KMP) Algorithm

Q1. Construct the prefix function using the Knuth-Morris-Pratt (KMP) algorithm for the pattern `ABABC`. Explain how you can use this prefix function to search for this pattern in the text `ABABABABC`. **(15 marks)** **[CSE 207 | L-2/T-1 | 2023–24]**

S7 — Randomized Algorithms

Monte Carlo Algorithms & Probability Amplification

Q1. Suppose you have designed a Monte Carlo Algorithm for a given problem which succeeds with probability $\geq \frac{1}{n^2}$. However, your boss has asked you to achieve at least $(1 - \frac{1}{n})$ success probability. Explain how you can achieve this. **(10 marks)** **[CSE 207 | L-2/T-1 | 2024–25]**

Las Vegas Algorithms & Randomized Quick Sort

Q1. Analyze the running time of the randomized quick sort algorithm. **(12 marks)** **[CSE 207 | L-2/T-1 | 2024–25]**